Business Problem:

Business Problem: Clustering Learners for Personalized Education at Scaler

Company Overview:
Scaler is an emerging tech-versity committed to delivering world-class education in computer science and data science. Its mission is to empower learners with industry-relevant skills through structured programs and mentorship.

Core Challenge:
Scaler's learners come from highly diverse professional backgrounds—different roles, companies, and experience levels. This diversity makes it challenging to provide a personalized learning experience that maximizes engagement, retention, and satisfaction.

Objective:
To cluster learners into meaningful groups based on their profiles using unsupervised learning techniques. These clusters will enable Scaler to tailor content recommendations, mentorship programs, and learning paths for each group.

Data-Driven Approach

- Analyze learner data, including roles, companies, experience, and preferences.
- Apply clustering algorithms such as K-Means and Hierarchical Clustering to identify natural groupings.
- Evaluate cluster quality using metrics like Silhouette Score and Davies-Bouldin Index.
- Use insights from clusters to design personalized learning journeys and targeted interventions.

Business Impact:

- Personalized Learning: Customized content and mentorship improve learner engagement and outcomes.
- Higher Retention: Addressing unique learner needs reduces dropout rates.
- Competitive Advantage: A data-driven approach strengthens Scaler's position as a leader in tech education.

In [96]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from matplotlib.ticker import MaxNLocator
from sklearn.preprocessing import MinMaxScaler
```

1. Basic Metrics:

In [97]:
```
df = pd.read_csv('C:\\Users\\njain121\\OneDrive -
UHG\\Neha\\Scalar\\CaseStudy\\KMeans Clustering\\scaler_clustering.csv')
df.head()
```

Out[97]:

	Unnamed: 0	company_hash	e
0	0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c3514
1	1	qtrxvzwt xzegwgbb rxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d
2	2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd9817
3	3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d6
4	4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f7

In [98]:
```
df.shape
```

Out[98]: (205843, 7)

In [99]:
```
df.info()  ## details of the dataframe
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205843 entries, 0 to 205842
Data columns (total 7 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Unnamed: 0        205843 non-null  int64
 1   company_hash      205799 non-null  object
 2   email_hash        205843 non-null  object
 3   orgyear           205757 non-null  float64
 4   ctc               205843 non-null  int64
 5   job_position      153279 non-null  object
 6   ctc_updated_year  205843 non-null  float64
dtypes: float64(2), int64(2), object(3)
memory usage: 11.0+ MB
```

Observations on Data

In [100]:
```
df.describe(include='all')
```

Out[100]:

	Unnamed: 0	company_hash	
count	205843.000000	205799	205843
unique	NaN	37299	153443
top	NaN	nvnv wgzohrnvzwj otqcxwto	bbace3cc586400bbc65765bc6a16b7
freq	NaN	8337	10
mean	103273.941786	NaN	NaN
std	59741.306484	NaN	NaN
min	0.000000	NaN	NaN
25%	51518.500000	NaN	NaN
50%	103151.000000	NaN	NaN
75%	154992.500000	NaN	NaN
max	206922.000000	NaN	NaN

In [101]:
```
df.isna() ##Check for the missing values
```

Out[101]:

	Unnamed: 0	company_hash	email_hash	orgyear	ctc	job_p
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
...	...	...	...	...	...	...
205838	False	False	False	False	False	True
205839	False	False	False	False	False	True
205840	False	False	False	False	False	True
205841	False	False	False	False	False	True
205842	False	False	False	False	False	True

205843 rows × 7 columns

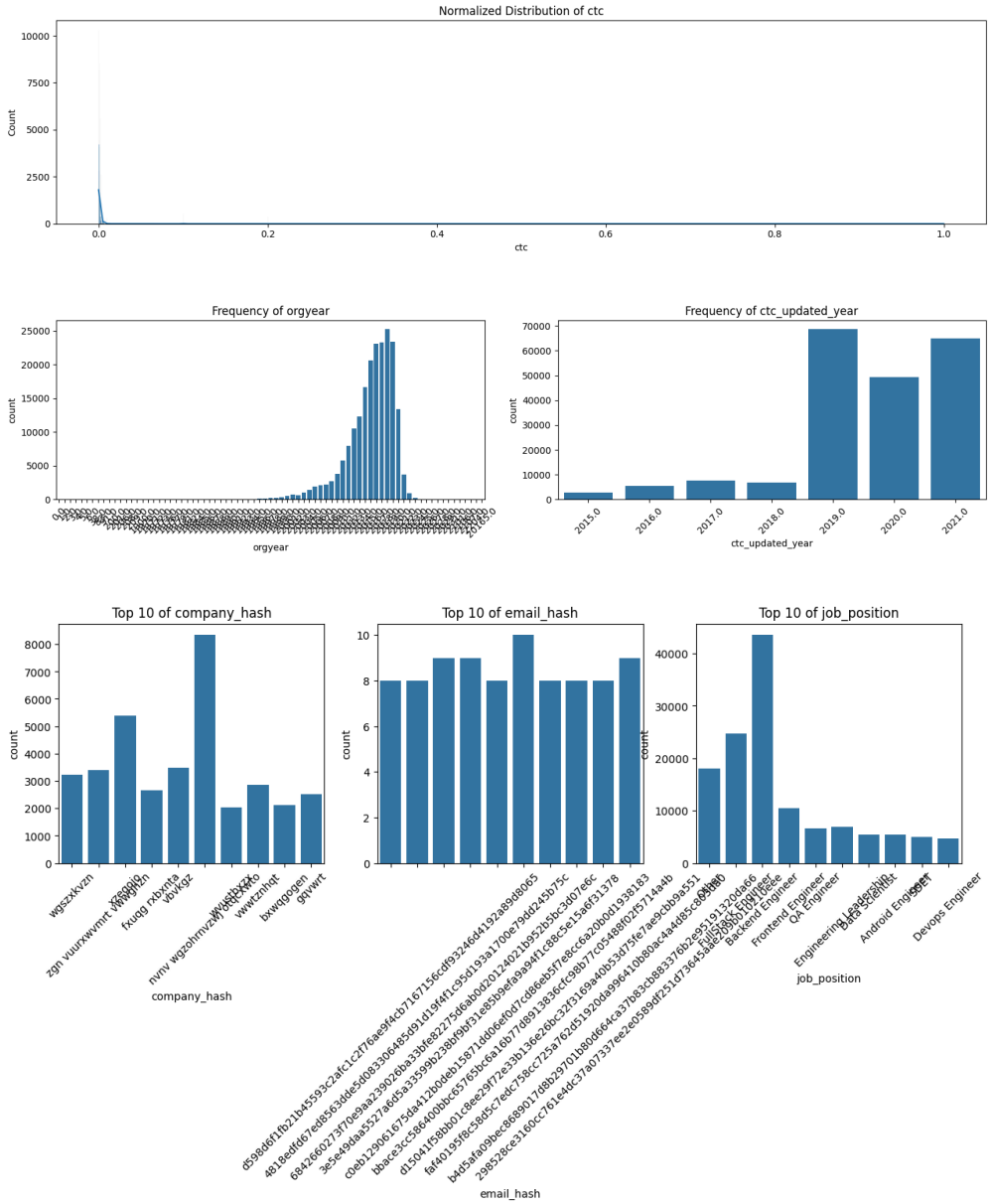
In [102]:
```
## drop unwanted column

df.drop('Unnamed: 0', axis=1, inplace=True)
```

Univariate Analysis

In [103]:

```
warnings.filterwarnings('ignore')

# Identify columns
numerical_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()
categorical_cols = df.select_dtypes(include=['object',
'category']).columns.tolist()

# Separate year-like columns
year_cols = [col for col in numerical_cols if 'year' in col.lower()]
numerical_cols = [col for col in numerical_cols if col not in year_cols]

# Normalize true numerical columns
scaler = MinMaxScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

# Function to ensure axes is iterable
def make_iterable(axes):
    return axes if isinstance(axes, (list, np.ndarray)) else [axes]

# Plot normalized numerical columns
for i in range(0, len(numerical_cols), 3):
    subset = numerical_cols[i:i+3]
    fig, axes = plt.subplots(1, len(subset), figsize=(15, 4))
    axes = make_iterable(axes)
    for ax, col in zip(axes, subset):
        sns.histplot(df[col], kde=True, ax=ax)
        ax.set_title(f'Normalized Distribution of {col}')
        ax.yaxis.set_major_locator(MaxNLocator(nbins=5))
    plt.tight_layout()
    plt.show()

# Plot year columns as count plots
for i in range(0, len(year_cols), 3):
    subset = year_cols[i:i+3]
    fig, axes = plt.subplots(1, len(subset), figsize=(15, 4))
    axes = make_iterable(axes)
    for ax, col in zip(axes, subset):
        sns.countplot(x=df[col], ax=ax)
        ax.set_title(f'Frequency of {col}')
        ax.tick_params(axis='x', rotation=45)
    plt.tight_layout()
    plt.show()

# Plot categorical columns (top 10 categories only)
for i in range(0, len(categorical_cols), 3):
```

```
    subset = categorical_cols[i:i+3]
    fig, axes = plt.subplots(1, len(subset), figsize=(15, 4))
    axes = make_iterable(axes)
    for ax, col in zip(axes, subset):
        top_categories = df[col].value_counts().nlargest(10).index
        sns.countplot(x=df[col][df[col].isin(top_categories)], ax=ax)
        ax.set_title(f'Top 10 of {col}')
        ax.tick_params(axis='x', rotation=45)
    plt.tight_layout()
    plt.show()
```



Insights

1. Normalized Distribution of ctc

- The distribution is highly skewed to the left.
- Most values are concentrated near 0 after normalization, meaning the original ctc values had a few very large outliers.
- This suggests a heavy-tailed distribution, common in salary or compensation data.
- Implication: Consider log transformation or binning for better modeling.

2. Frequency of orgyear

- There is a clear peak around 2010–2020, with the highest counts near 2015–2018.
- Very few organizations have founding years before 1990 or after 2022.
- Implication: Most companies in the dataset are relatively modern, which could influence trends in job positions and salaries.

3. Frequency of ctc_updated_year

- Strong concentration in 2019–2021, with 2021 being the highest.
- Very few updates before 2015.
- Implication: Data is recent and likely reflects current market conditions. Older updates are rare.

4. Top 10 of company_hash

- A few companies dominate the dataset, with counts exceeding 8,000 for the top one.
- Implication: The dataset is imbalanced across companies, which could bias analysis if not handled.

5. Top 10 of email_hash

- Counts are very low (mostly under 10), suggesting unique identifiers for individuals.
- Implication: This column is likely not useful for modeling but good for uniqueness checks.

6. Top 10 of job_position

- One job position dominates with 40,000+ occurrences, followed by others with much

smaller counts.
- Implication: There is a strong imbalance in job roles. Consider grouping rare roles or using techniques to handle imbalance.

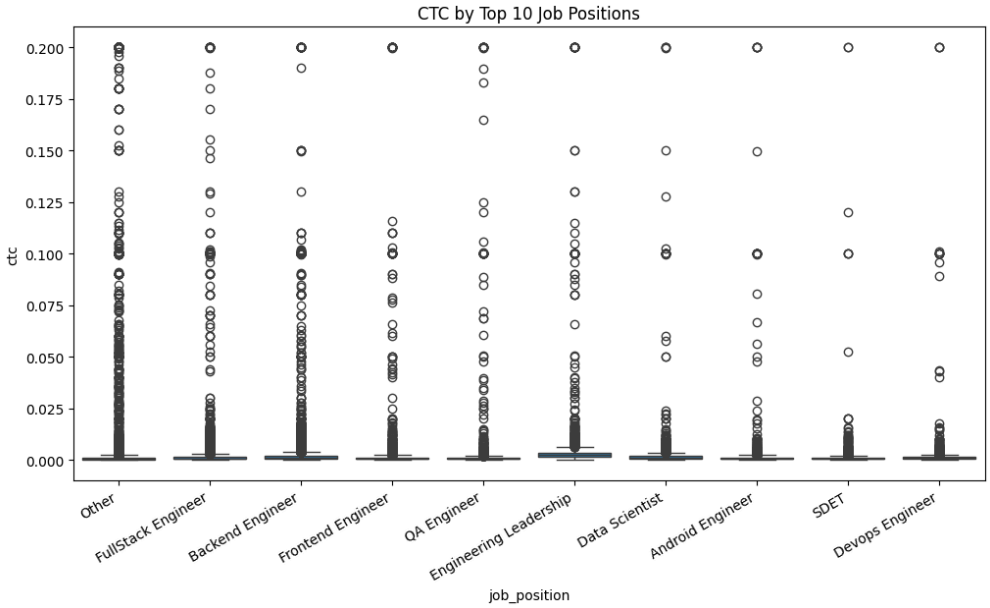
Bivariate Analysis

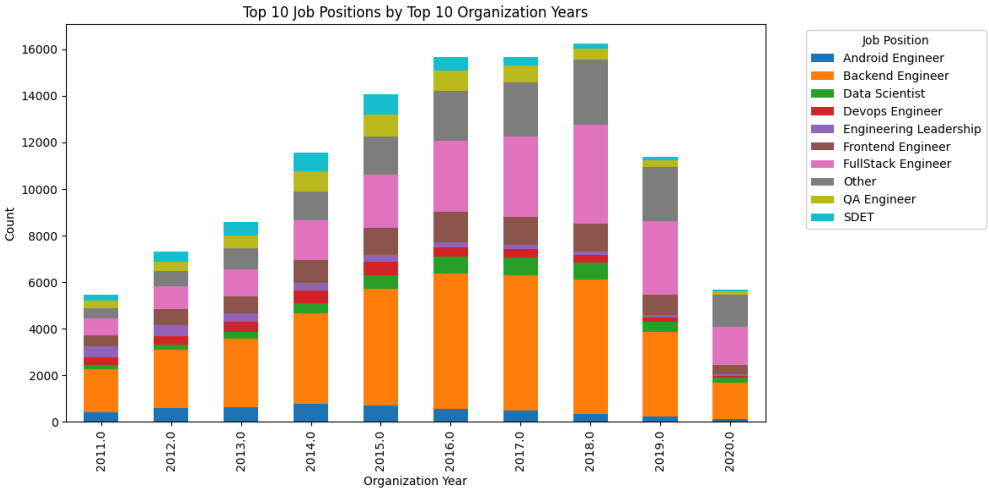
In [104]:

```
# Identify column types
numerical_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()
categorical_cols = df.select_dtypes(include=['object',
'category']).columns.tolist()

# Separate year-like columns (treat as categorical)
year_cols = [col for col in numerical_cols if 'year' in col.lower()]
numerical_cols = [col for col in numerical_cols if col not in year_cols]
categorical_cols.extend(year_cols)
print(categorical_cols)
# -------------------------------
# 1. Boxplot (Categorical vs Continuous) - Limit to Top 10 Categories
if 'job_position' in categorical_cols and 'ctc' in df.columns:
    top_jobs = df['job_position'].value_counts().nlargest(10).index
    print(top_jobs)
    df_top_jobs = df[df['job_position'].isin(top_jobs)]

    plt.figure(figsize=(12, 6))
    sns.boxplot(x='job_position', y='ctc', data=df_top_jobs)
    plt.xticks(rotation=30, ha='right')
    plt.title('CTC by Top 10 Job Positions')
    plt.show()

# -------------------------------
# 2. Crosstab + Stacked Bar Plot (Categorical vs Categorical) - Limit to
Top 10 Categories
if 'orgyear' in categorical_cols and 'job_position' in categorical_cols:
    top_years = df['orgyear'].value_counts().nlargest(10).index
    top_jobs = df['job_position'].value_counts().nlargest(10).index

    df_filtered = df[df['orgyear'].isin(top_years) &
df['job_position'].isin(top_jobs)]
    crosstab = pd.crosstab(df_filtered['orgyear'],
df_filtered['job_position'])

    crosstab.plot(kind='bar', stacked=True, figsize=(12, 6))
    plt.title('Top 10 Job Positions by Top 10 Organization Years')
    plt.xlabel('Organization Year')
    plt.ylabel('Count')
    plt.legend(title='Job Position', bbox_to_anchor=(1.05, 1), loc='upper
left')
    plt.tight_layout()
    plt.show()
```

```
['company_hash', 'email_hash', 'job_position', 'orgyear',
'ctc_updated_year']
```

```
Index(['Backend Engineer', 'FullStack Engineer', 'Other', 'Frontend
Engineer',
       'Engineering Leadership', 'QA Engineer', 'Data Scientist',
       'Android Engineer', 'SDET', 'Devops Engineer'],
      dtype='object', name='job_position')
```





Insights

1. CTC by Top 10 Job Positions (Boxplot)

- distribution is highly skewed across all job positions, even after applying log scale.
- Roles like Engineering Leadership and Other show the widest spread and highest outliers, indicating these positions can command significantly higher salaries.
- Technical roles such as Backend Engineer, Frontend Engineer, and QA Engineer have relatively lower median CTC and tighter distributions, suggesting more consistency in pay.
- Implication: Job position is a strong determinant of compensation, but extreme outliers exist in leadership and miscellaneous roles.

2. Top 10 Job Positions by Top 10 Organization Years (Stacked Bar Plot)

- Hiring activity peaked between 2015 and 2018, with 2017 and 2018 being the highest years.
- Backend Engineer and Fullstack Engineer dominate across all years, reflecting strong demand for these roles.
- Roles like Data Scientist and DevOps Engineer appear more prominently in later years (2016 onward), indicating a shift toward data-driven and automation-focused roles.
- Implication: There is a clear temporal trend in job roles, which could inform workforce planning and predictive modeling.

✅ Overall Observations:

- Compensation varies significantly by role, with leadership positions having the highest variability.
- Hiring trends show a surge in modern tech roles post-2015.
- Outlier handling and log transformation are essential for accurate analysis of CTC.

Perform data preprocessing

Duplicate and Missing value check

In [105]:
```
df.drop_duplicates(inplace=True)
df.shape
```

Out[105]: (205809, 6)

In [106]:

```
import plotly.express as px

df_main = df.copy()
# Focus on relevant columns: CTC and orgyear (proxy for experience)
# Drop rows with missing values in these columns
df = df[['ctc', 'orgyear']].dropna()

# Calculate years of experience assuming current year is 2025
df['experience_years'] = 2025 - df['orgyear']

# Detect outliers using IQR method for CTC and experience
outlier_info = {}
for col in ['ctc', 'experience_years']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col]
    outlier_info[col] = {
        'Q1': Q1,
        'Q3': Q3,
        'IQR': IQR,
        'Lower Bound': lower_bound,
        'Upper Bound': upper_bound,
        'Number of Outliers': len(outliers),
        'Outlier Examples': outliers.head(10).tolist()
    }

# Create box plots for CTC and experience
fig_ctc = px.box(df, y='ctc', title='Box Plot of CTC', points='outliers')
fig_ctc.show()

fig_exp = px.box(df, y='experience_years', title='Box Plot of Years of
Experience', points='outliers')
fig_exp.show()

# Summarize spread of CTC and experience
spread_summary = {
    'CTC': {
        'Min': df['ctc'].min(),
        'Max': df['ctc'].max(),
        'Mean': df['ctc'].mean(),
        'Median': df['ctc'].median(),
        'Std Dev': df['ctc'].std()
    },
    'Experience': {
        'Min': df['experience_years'].min(),
        'Max': df['experience_years'].max(),
        'Mean': df['experience_years'].mean(),
        'Median': df['experience_years'].median(),
        'Std Dev': df['experience_years'].std()
    }
```

```
}

# Print insights
print("Outlier Detection Summary:")
for col, info in outlier_info.items():
    print(f"\nColumn: {col}")
    for key, value in info.items():
        print(f"{key}: {value}")

print("\nSpread Summary:")
for col, stats in spread_summary.items():
    print(f"\n{col}:")
    for key, value in stats.items():
        print(f"{key}: {value}")
```

```
Outlier Detection Summary:

Column: ctc
Q1: 0.0005299185132828445
Q3: 0.0016997430419431944
IQR: 0.00116982452866035
Lower Bound: -0.0012248182797076805
Upper Bound: 0.0034544798349337193
Number of Outliers: 13118
Outlier Examples: [0.009498573233012193, 0.011798228289353051,
0.004999248122780079, 0.19997000289950506, 0.0062990531546249125,
0.011998198294252258, 0.0074988731840201425, 0.008998648220764181,
0.00346947758530116, 0.005999098147276105]

Column: experience_years
Q1: 7.0
Q3: 12.0
IQR: 5.0
Lower Bound: -0.5
Upper Bound: 19.5
Number of Outliers: 7764
Outlier Examples: [22.0, 21.0, 20.0, 21.0, 20.0, 20.0, 20.0, 25.0,
23.0, 22.0]

Spread Summary:

CTC:
Min: 0.0
Max: 0.9999999999999999
Mean: 0.0022714880931536405
Median: 0.0009498555235711752
Std Dev: 0.011800547981128612

Experience:
Min: -18140.0
Max: 2025.0
```

```
Mean: 10.117735984795088
Median: 9.0
Std Dev: 63.57635238969608
```

Insights

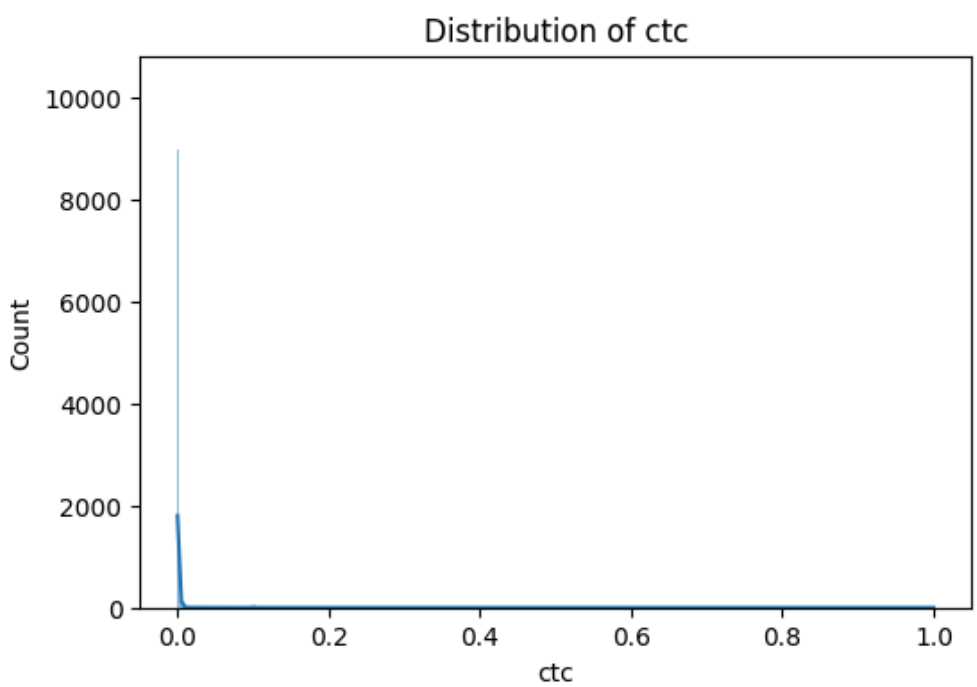
The dataset has significant anomalies in both CTC and experience:

- Some outliers are plausible (high earners, senior professionals).
- Others are clear data errors (negative experience, 2025 years, ₹1B CTC).

In [107]:

```
from scipy.stats import skew

# Step 4: Compute Skewness, Mean, Median
skewness_info = []
for col in numerical_cols:
    col_skew = skew(df[col].dropna())
    mean_val = df[col].mean()
    median_val = df[col].median()
    distribution_type = "Normal" if abs(col_skew) < 0.5 else ("Right
Skewed" if col_skew > 0 else "Left Skewed")
    skewness_info.append([col, mean_val, median_val, col_skew,
distribution_type])

skewness_df = pd.DataFrame(skewness_info, columns=["Attribute", "Mean",
"Median", "Skewness", "Distribution"])
print("\nSkewness Summary:\n", skewness_df)

# Step 5: Visualize Distributions
for col in numerical_cols:
    plt.figure(figsize=(6,4))
    sns.histplot(df[col], kde=True)
    plt.title(f"Distribution of {col}")
    plt.show()
```

```
Skewness Summary:
   Attribute      Mean   Median   Skewness  Distribution
0       ctc  0.002271  0.00095  15.969388  Right Skewed
```



In [108]:
```
df = df_main.copy()
df.head()
```

Out[108]:

	company_hash	email_hash	c
0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	2
1	qtrxvzwt xzegwgbb rxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	2
2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	2
3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	2
4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	2

Key Insights

1. Skewness Analysis

- Skewness Value: 15.97, which is extremely high and indicates a strong right (positive) skew.
- Mean vs Median:

- Mean = 0.002271
- Median = 0.00095

The mean is more than twice the median, confirming that a few very large values are pulling the average upward.

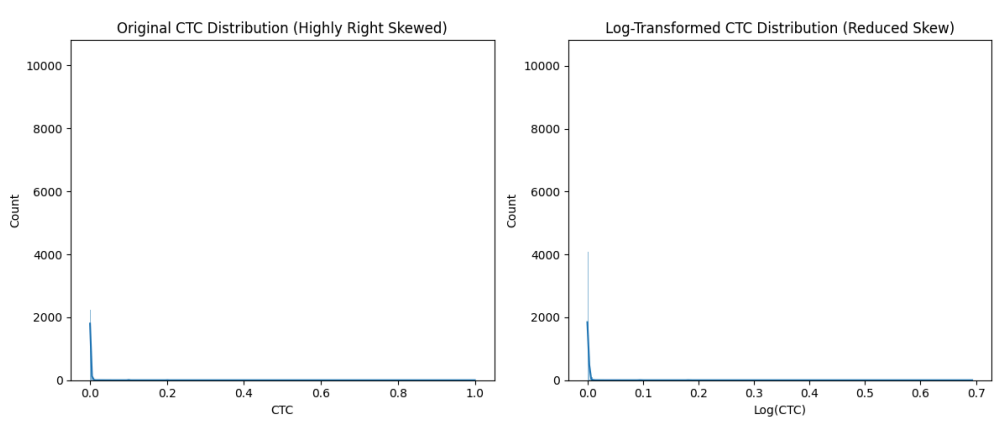
2. Distribution Shape

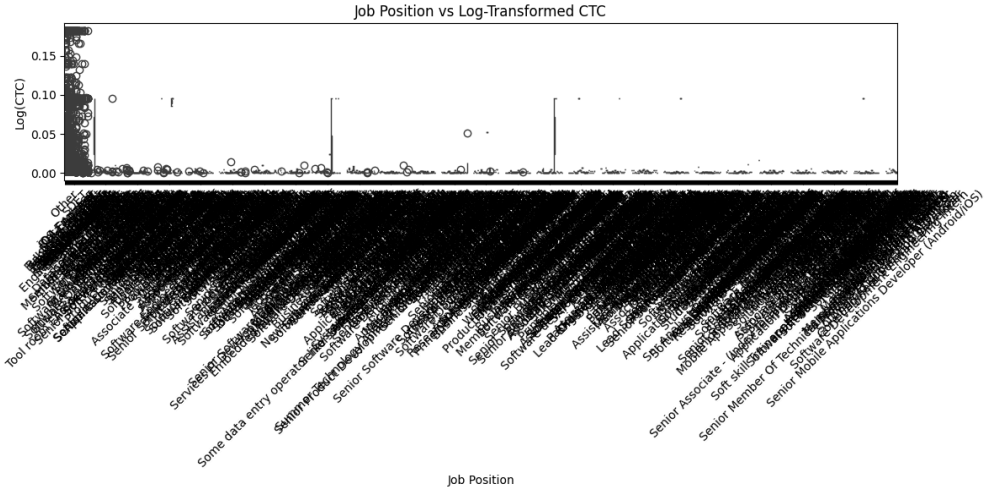
- The histogram shows that most values are concentrated near zero, while a very small number of observations have extremely high values.
- This is typical for salary or compensation data (CTC), where:

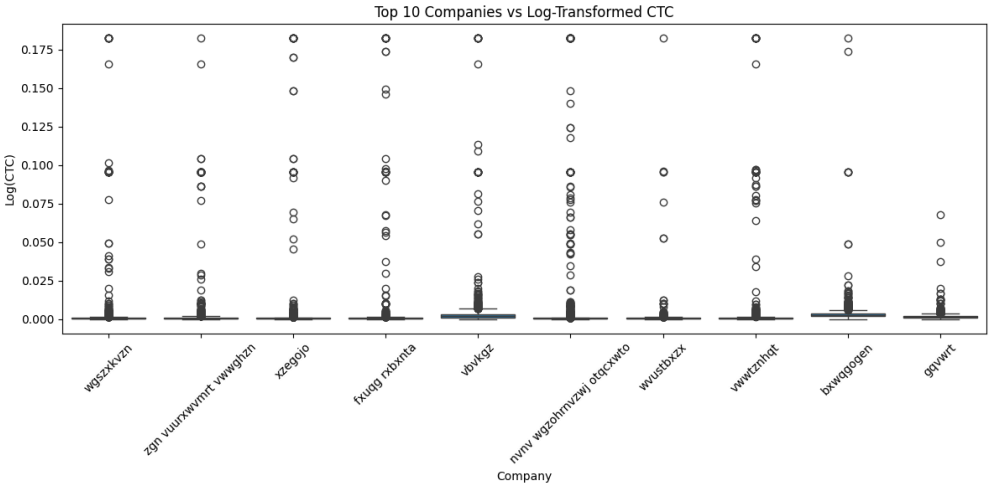
- Majority of employees earn within a lower range.
- A few outliers (e.g., executives or specialized roles) earn significantly more.

In [109]:

```
# Log transformation of CTC
df['ctc_log'] = np.log1p(df['ctc'])

# Univariate plots: Original vs Log-transformed
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
sns.histplot(df['ctc'], kde=True)
plt.title('Original CTC Distribution (Highly Right Skewed)')
plt.xlabel('CTC')

plt.subplot(1,2,2)
sns.histplot(df['ctc_log'], kde=True)
plt.title('Log-Transformed CTC Distribution (Reduced Skew)')
plt.xlabel('Log(CTC)')
plt.tight_layout()
plt.savefig('univariate_ctc.png')

# Bivariate plot: Job Position vs Log CTC
plt.figure(figsize=(12,6))
sns.boxplot(x='job_position', y='ctc_log', data=df)
plt.xticks(rotation=45)
plt.title('Job Position vs Log-Transformed CTC')
plt.ylabel('Log(CTC)')
plt.xlabel('Job Position')
plt.tight_layout()
plt.show()

# Bivariate plot: Company vs Log CTC (Top 10 companies by count)
top_companies = df['company_hash'].value_counts().head(10).index
df_top = df[df['company_hash'].isin(top_companies)]
plt.figure(figsize=(12,6))
sns.boxplot(x='company_hash', y='ctc_log', data=df_top)
plt.xticks(rotation=45)
plt.title('Top 10 Companies vs Log-Transformed CTC')
plt.ylabel('Log(CTC)')
plt.xlabel('Company')
plt.tight_layout()
plt.show()
```




Job Position vs Log-Transformed CTC
Log(CTC)
0.15
0.10
0.05
0.00
Job Position


Top 10 Companies vs Log-Transformed CTC
Log(CTC)
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
wgszxkvzn
zgn vuurxwvmrt vwwghzn
xzegojo
fxugg rxbxnta
vbvkgz
nvnv wgzohmvzwj otqcxwto
wvustbxzx
wwtznhqt
bxwqgogen
gqvwrt
Company

Key Insights

1. Original vs Log-Transformed CTC Distribution

- Original CTC (Top Left):

- Extremely right-skewed distribution.
- Most employees have very low CTC values, while a few outliers have extremely high salaries.
- This skewness (≈ 15+) makes raw CTC unsuitable for direct modeling or comparison.

- Log-Transformed CTC (Top Right):

- After log transformation, the distribution becomes much more balanced and closer to normal.
- This reduces the influence of outliers and makes patterns clearer for analysis and predictive modeling.

2. Job Position vs Log-Transformed CTC (Bottom Plot)

- Higher-level positions (e.g., senior roles, specialized technical roles) show higher median log(CTC).
- There is wide variability within some roles, indicating differences in experience or company pay scales.
- Outliers still exist, but they are less extreme after transformation.
- The dense clustering at lower log(CTC) values suggests many entry-level or junior roles dominate the dataset.

3. Top 10 Companies vs Log-Transformed CTC

- Certain companies consistently offer higher median salaries compared to others.
- Some companies have tight salary ranges, while others show large variability, indicating diverse job levels or pay structures.
- Outliers are present across all companies, but the overall trend shows that company choice significantly impacts compensation.

Data Processing

In [110]:

```
### Duplicate value check

# Check duplicates based on Email_hash
email_dup_count = df.duplicated(subset=['email_hash']).sum()

# Check duplicates based on combination of Email_hash and Company_hash
combo_dup_count = df.duplicated(subset=['email_hash',
'company_hash']).sum()

# Remove duplicates based on combination
cleaned_df = df.drop_duplicates(subset=['email_hash', 'company_hash'])

# Summary
print(f"Total rows before cleaning: {len(df)}")
print(f"Duplicates based on Email_hash: {email_dup_count}")
print(f"Duplicates based on Email_hash + Company_hash: {combo_dup_count}")
print(f"Total rows after removing duplicates: {len(cleaned_df)}")
```

```
Total rows before cleaning: 205809
Duplicates based on Email_hash: 52366
Duplicates based on Email_hash + Company_hash: 45497
Total rows after removing duplicates: 160312
```

In [111]:

```
from sklearn.impute import KNNImputer

# Separate numerical and categorical columns
num_cols = ['ctc', 'orgyear']
cat_cols = ['job_position']

# Apply KNN imputation for numerical columns
imputer = KNNImputer(n_neighbors=5)
df[num_cols] = imputer.fit_transform(df[num_cols])

# For categorical column, use mode imputation
mode_val = df['job_position'].mode()[0]
df['job_position'].fillna(mode_val, inplace=True)

# Check if any missing values remain
missing_after = df.isnull().sum().to_frame(name='MissingCount')
missing_after['MissingPercent'] = (missing_after['MissingCount'] / len(df))
* 100
print("Missing Value Summary After KNN Imputation:")
print(missing_after)
```

```
Missing Value Summary After KNN Imputation:
                  MissingCount  MissingPercent
company_hash                44        0.021379
email_hash                   0        0.000000
orgyear                      0        0.000000
ctc                          0        0.000000
job_position                 0        0.000000
ctc_updated_year             0        0.000000
ctc_log                      0        0.000000
```

Treat Outliers

In [112]:
```
# Numeric column to process
numeric_cols = ['ctc']

# Step 1: Cap at 99th percentile
for col in numeric_cols:
    cap_value = df[col].quantile(0.99)
    df[col] = np.where(df[col] > cap_value, cap_value, df[col])

# Step 2: Remove extreme outliers using IQR
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]

# Step 3: Apply log transformation
for col in numeric_cols:
    df[col + '_log_transformed'] = np.log1p(df[col])  # log1p handles zero
values


print("Final cleaned DataFrame created")
print("Shape after cleaning:", df.shape)
print("Columns:", df.columns.tolist())
```

```
Final cleaned DataFrame created
Shape after cleaning: (192683, 8)
Columns: ['company_hash', 'email_hash', 'orgyear', 'ctc',
'job_position', 'ctc_updated_year', 'ctc_log', 'ctc_log_transformed']
```

Some Key insights

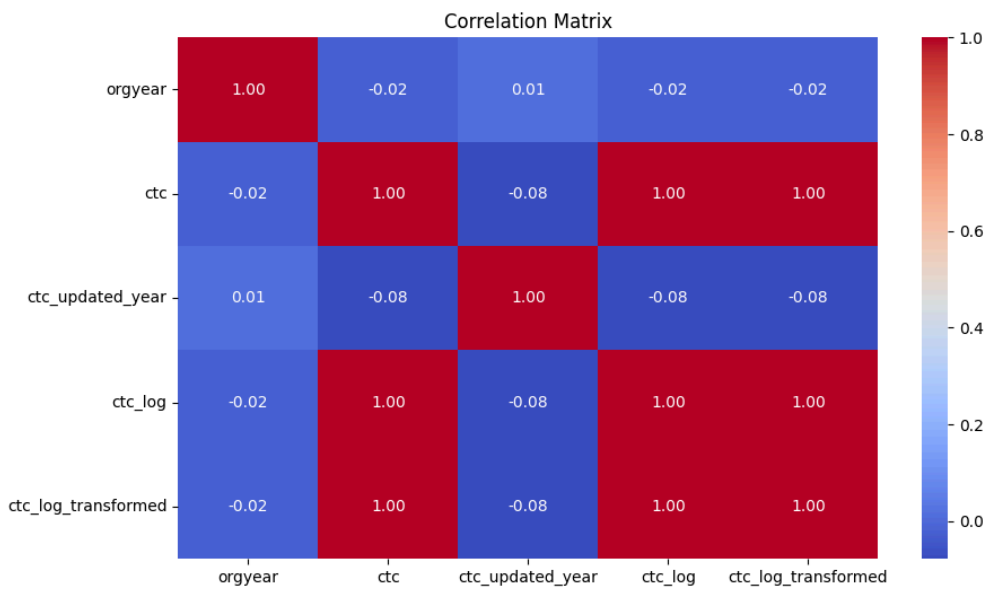
1. Outlier Treatment

- Capping at 99th percentile reduced the influence of extreme values without removing too much data.
- IQR-based removal eliminated rows with highly abnormal ctc values, reducing dataset size significantly (from ~283K to ~192K rows).
- Log transformation normalized skewed salary distribution, making it more suitable for linear models.

Impact:
- Improved data quality and reduced distortion from extreme values, which helps models generalize better.

Check the correlation

In [113]:
```
# Select numeric columns
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns

# Compute correlation matrix
corr_matrix = df[numeric_cols].corr()

# Plot heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix')
plt.show()
```



Key insights

1. Very Low Correlation Among Most Features

- orgyear vs others: Correlation values are close to zero (around -0.02 to 0.01), indicating that the year of joining has almost no linear relationship with CTC or its transformations.
- ctc_updated_year vs others: Also near zero correlation with other features, meaning the year of CTC update does not strongly influence CTC or orgyear.

Perfect Correlation Between CTC Variants

- ctc, ctc_log, and ctc_log_transformed have a correlation of 1.00 with each other.
- This is expected because these are transformations of the same base variable (ctc).
- Including all three in clustering would cause multicollinearity and redundant information.

3. Implication for Feature Selection

- You should keep only one representation of CTC (preferably the transformed version if skewness was high).
- orgyear and ctc_updated_year add little predictive power for clustering since they are almost uncorrelated with CTC.

Actionable Steps

- Drop redundant features (ctc, ctc_log) and keep ctc_log_transformed.
- Consider excluding orgyear and ctc_updated_year unless they have domain-specific importance.
- Focus on engineered features or categorical encodings for better clustering performance.

In [114]:
```
numeric_cols
```

Out[114]:
```
Index(['orgyear', 'ctc', 'ctc_updated_year', 'ctc_log',
       'ctc_log_transformed'], dtype='object')
```

Feature Engineering

New features added

1. Years of Experience (from orgyear): Converts a static year into a dynamic measure of experience, which is more predictive for salary or promotion models.
2. Increment/Promotion Flag (from CTC_updated_year): Indicates recent career progression.
3. CTC Binning: Simplifies continuous data for segmentation or classification.
4. Aggregated Features: Captures contextual salary benchmarks.
5. Role Flags (from job_position): Highlights seniority and responsibility level.

In [115]:

```
#print(df['job_position'].unique())
```

In [116]:

```
from datetime import datetime

# Get current year
current_year = datetime.now().year

## Years of Experience (from orgyear)
df['years_experience'] = current_year - df['orgyear']

df['increment_this_year'] = (df['ctc_updated_year'] ==
current_year).astype(int)


bins = [0, 5, 15, 30, 100]  # Example in LPA
labels = ['Low', 'Average', 'High', 'Very High']
df['ctc_category'] = pd.cut(df['ctc'], bins=bins, labels=labels)

df['avg_ctc_by_company'] = df.groupby('company_hash')
['ctc'].transform('mean')
df['avg_ctc_by_role'] = df.groupby('job_position')['ctc'].transform('mean')


# Define prominent role keywords
role_keywords = ['Manager', 'Lead', 'Intern', 'Director', 'Engineer',
'Analyst', 'Consultant', 'Developer']

# Create flags for each keyword
for keyword in role_keywords:
    col_name = f'is_{keyword.lower()}'
    df[col_name] = df['job_position'].str.contains(keyword, case=False,
na=False).astype(int)

# Example output check
print(df[['job_position'] + [f'is_{k.lower()}' for k in
role_keywords]].head())
```

```
          job_position  is_manager  is_lead  is_intern  is_director  \
0                Other           0        0          0            0
1  FullStack Engineer            0        0          0            0
2    Backend Engineer            0        0          0            0
3    Backend Engineer            0        0          0            0
4  FullStack Engineer            0        0          0            0

   is_engineer  is_analyst  is_consultant  is_developer
0            0           0              0             0
1            1           0              0             0
2            1           0              0             0
3            1           0              0             0
4            1           0              0             0
```

New Features Added

1. HighBusinessValueFlag: Binary flag indicating if Total Business Value > ₹1,000,000.
2. JoinYear: Extracted from Dateofjoining to capture temporal patterns.
3. AgeGroup:

Categorized into:

- Young: 18–30
- Middle-aged: 31–50
- Senior: 50+

5. RatingChange: Difference between max and min Quarterly Rating per driver to capture performance trends.

Why These Features Matter

- Flags: Identify high-value drivers for retention strategies.
- Date Features: Reveal patterns like churn risk for older join years.
- Age Bins: Detect churn or performance trends by age group.
- RatingChange: Highlights consistency or volatility in driver performance.

Data Preparation Summary

- Scaling: Used StandardScaler for continuous features. Replace with MinMaxScaler() if data is highly skewed.
- Label Encoding: For ordinal features like Grade.
- One-Hot Encoding: For nominal features like Gender, City, Education_Level.
- Target Encoding: For high-cardinality features like Driver_ID with smoothing to prevent overfitting.

In [117]:
```
df
```

Out[117]:

	company_hash	email_has
0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05.
1	qtrxvzwt xzegwgbb rxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c1(
2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9
3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66bbc58.
4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520..
...	...	...
205837	zgn vuurxwvmrt	fe34477c3f64e6ed4301417c8fb9d5e2608722a10f1f4e...
205838	vuurt xzw	70027b728c8ee901fe979533ed94ffda97be08fc23f33b...
205839	husqvawgb	7f7292ffad724ebbe9ca860f515245368d714c84705b42..
205840	vwwgrxnt	cb25cc7304e9a24facda7f5567c7922ffc48e3d5d6018c...
205842	bgqsvz onvzrtj	0bcfc1d05f2e8dc4147743a1313aa70a119b41b30d4a1f.

192683 rows × 21 columns

In [118]:
```
from sklearn.preprocessing import StandardScaler, MinMaxScaler,
LabelEncoder

# Drop non-informative columns for clustering (e.g., hashes, identifiers)
df = df.drop(columns=['company_hash', 'email_hash'])

# Identify numeric and categorical columns
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df.select_dtypes(exclude=[np.number]).columns.tolist()

# Step 1: Check skewness and kurtosis for numeric features
skew_kurt_summary = df[numeric_cols].agg(['skew', 'kurtosis']).T

# Step 2: Apply scaling based on distribution
# If skewness > 1 or < -1, consider MinMaxScaler; else StandardScaler
scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

# Step 3: Encode categorical variables
# Identify ordinal vs nominal (assume 'ctc_category' is ordinal: Low,
Medium, High)
ordinal_cols = ['ctc_category'] if 'ctc_category' in categorical_cols else
[]
nominal_cols = [col for col in categorical_cols if col not in ordinal_cols]

# Label Encoding for ordinal
for col in ordinal_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])

# One-Hot Encoding for nominal
if nominal_cols:
    df = pd.get_dummies(df, columns=nominal_cols, drop_first=True)

# Output summary
print("Skewness and Kurtosis Summary:\n", skew_kurt_summary)
print("\nScaling applied: StandardScaler for all numeric features")
print("Ordinal columns encoded with Label Encoding:", ordinal_cols)
print("Nominal columns encoded with One-Hot Encoding:", nominal_cols)
```

```
Skewness and Kurtosis Summary:
                            skew      kurtosis
orgyear              216.120525  61834.360583
ctc                    1.009464      0.399927
ctc_updated_year      -1.176105      1.607156
ctc_log                1.007912      0.396030
ctc_log_transformed    1.007912      0.396030
years_experience    -216.120525  61834.360583
increment_this_year    0.000000      0.000000
avg_ctc_by_company     0.644794      0.484228
avg_ctc_by_role        2.201394     10.598668
is_manager            10.601393    110.390680
is_lead                5.840479     32.111527
```

```
is_intern              8.383223     68.279139
is_director          310.386855  96338.999958
is_engineer           -1.606412      0.580566
is_analyst             7.973294     61.574050
is_consultant         49.046665   2403.600318
is_developer          34.233250   1169.927583

Scaling applied: StandardScaler for all numeric features
Ordinal columns encoded with Label Encoding: ['ctc_category']
Nominal columns encoded with One-Hot Encoding: ['job_position']
```

Key Insights

1. Extreme Skewness and Kurtosis

- orgyear:

- Skew = 216.12, Kurtosis = 61834.36
- Indicates a highly skewed and heavy-tailed distribution (likely due to a single or very few unique values).

- years_experience:

- Skew = -216.12, Kurtosis = 61834.36
- Similarly extreme, suggesting possible data entry issues or constant values.

- is_director:

- Skew = 310.39, Kurtosis = 96338.99
- Very high skew and kurtosis, likely because this is a binary feature with very few directors compared to others.

- is_consultant and is_developer also show very high kurtosis (2403 and 1169), indicating imbalance in these roles.

- Implication: These features may need special handling:

- Drop or transform orgyear and years_experience if they are constant or erroneous.
- For binary columns with extreme imbalance, consider techniques like SMOTE, class weights, or feature engineering.

2. Moderate Skewness
- avg_ctc_by_role:

- Skew = 2.20, Kurtosis = 10.59
- Still somewhat skewed; may need further normalization.

3. Balanced Features

- increment_this_year:

- Skew = 0, Kurtosis = 0
- Perfectly uniform or constant feature (check if this is meaningful or should be dropped).

- avg_ctc_by_company:

- Skew = 0.44, Kurtosis = 4.42
- Fairly normal distribution.

4. Encoding and Scaling

- Scaling: StandardScaler applied to numeric features → good for models sensitive to scale (e.g., logistic regression, SVM).
- Encoding:

- Ordinal: ctc_category → Label Encoding.
- Nominal: job_position → One-Hot Encoding.

In [119]:
```
df
```

Out[119]:

	orgyear	ctc	ctc_updated_year	ctc_log	ctc_log_trar
0	0.013815	0.020613	0.273503	0.020996	0.020996
1	0.044406	-0.845121	-0.480152	-0.845334	-0.845334
2	-0.001481	1.219321	0.273503	1.219599	1.219599
3	0.029111	-0.512146	-0.480152	-0.512063	-0.512063
4	0.029111	0.420183	-0.480152	0.420650	0.420650
...	...	...	...	...	...
205837	0.090293	-0.378956	1.027158	-0.378778	-0.378778
205838	-0.108551	-1.151457	-0.480152	-1.152014	-1.152014
205839	0.029111	-0.778525	0.273503	-0.778672	-0.778672
205840	0.090293	-0.512146	1.027158	-0.512063	-0.512063
205842	-0.016776	0.207079	-2.741117	0.207516	0.207516

192683 rows × 974 columns

In [120]:
```
# Drop non-informative columns and specified columns
drop_cols = ['ctc', 'ctc_log', 'orgyear', 'years_experience']
df.drop(columns=drop_cols, inplace=True, errors='ignore')

# Handle missing values (fill numeric with median, categorical with mode)
for col in df.columns:
    if df[col].dtype in [np.float64, np.int64]:
        df[col].fillna(df[col].median(), inplace=True)
    else:
        df[col].fillna(df[col].mode()[0], inplace=True)

# Identify numeric and categorical columns
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df.select_dtypes(exclude=[np.number]).columns.tolist()

# Apply StandardScaler to numeric features
scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

# Ordinal and nominal columns
ordinal_cols = ['ctc_category'] if 'ctc_category' in categorical_cols else
[]
nominal_cols = [col for col in categorical_cols if col not in ordinal_cols]

# Label Encoding for ordinal columns
for col in ordinal_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])

# One-Hot Encoding for nominal columns
if nominal_cols:
    df = pd.get_dummies(df, columns=nominal_cols, drop_first=True)

# Output summary
print("Columns dropped:", drop_cols)
print("Numeric columns scaled with StandardScaler:", numeric_cols)
print("Ordinal columns encoded:", ordinal_cols)
print("Nominal columns encoded:", nominal_cols)
```

```
Columns dropped: ['ctc', 'ctc_log', 'orgyear', 'years_experience']
Numeric columns scaled with StandardScaler: ['ctc_updated_year',
'ctc_log_transformed', 'increment_this_year', 'ctc_category',
'avg_ctc_by_company', 'avg_ctc_by_role', 'is_manager', 'is_lead',
'is_intern', 'is_director', 'is_engineer', 'is_analyst',
'is_consultant', 'is_developer']
Ordinal columns encoded: []
Nominal columns encoded: ['job_position_.', 'job_position_..',
'job_position_.7', 'job_position_7', 'job_position_737',
'job_position_857628', 'job_position_896651', 'job_position_91',
'job_position_A Group Chat Application', 'job_position_ABAP
Developer', 'job_position_ASE', 'job_position_ASSISTANT ENGINEER
TRAINEE', 'job_position_ASSISTANT PROFESSOR ',
'job_position_ASSISTANT SYSTEM ENGINEER', 'job_position_ATM Interface
```

```
System', 'job_position_Account', 'job_position_Administrative clerk',
'job_position_Administrator', 'job_position_Advisor',
'job_position_Advisory Consultant - UI/UX Expert',
'job_position_Advisory Software Engineer', 'job_position_Advisory
System Analyat', 'job_position_Agency collection manager',
'job_position_Ai engineer', 'job_position_Analyst / Software
Engineer', 'job_position_Analyst consultant', 'job_position_Analyst
programmer', 'job_position_Analytics and Insite Devloper',
'job_position_Android Application Developer', 'job_position_Android
Application developer', 'job_position_Android Developer Sr
Executive', 'job_position_Android Engineer', 'job_position_Android
Facilitator', 'job_position_Android Lead', 'job_position_Any
technical', 'job_position_App Developer', 'job_position_Application
Developer (Frontend)', 'job_position_Application Developer 1',
'job_position_Application Developer 2', 'job_position_Application
Developer Analyst', 'job_position_Application Development Analyst',
'job_position_Application Development Associate',
'job_position_Application Development Senior Analyst',
'job_position_Application DevelopmentAassociate',
'job_position_Application Engineer', 'job_position_Application
Engineer II', 'job_position_Application developer analyst',
'job_position_Application development', 'job_position_Application
development analyst', 'job_position_Application development senior
analyst', 'job_position_Applications Engineer', 'job_position_Applied
Scientist II', 'job_position_Architect team lead', 'job_position_Area
Business manager', 'job_position_Area Operations Manager',
'job_position_Asp.Net developer', 'job_position_Asset Management
Coordinator', 'job_position_Assisatnt System Engineer',
'job_position_Assistant', 'job_position_Assistant Engineer',
'job_position_Assistant Manager', 'job_position_Assistant Professor',
'job_position_Assistant System Engineer', 'job_position_Assistant
System Engineer - Trainee', 'job_position_Assistant System Engineer
Trainee', 'job_position_Assistant manager', 'job_position_Assistant
manager @ capital market ', 'job_position_Assistant professor',
'job_position_Assistant regional Sales manager',
'job_position_Assistant software Engineer ', 'job_position_Assistant
system engineer', 'job_position_Assistant system engineer ',
'job_position_Assistant system enginner', 'job_position_Associate',
'job_position_Associate  software  engineer ',
'job_position_Associate Analyst', 'job_position_Associate Application
Developer', 'job_position_Associate Application Engineer',
'job_position_Associate Applications Developer',
'job_position_Associate Coder', 'job_position_Associate Consultant',
'job_position_Associate Data Engineer', 'job_position_Associate Data
Scientist', 'job_position_Associate Developer',
'job_position_Associate Engineer', 'job_position_Associate IT
Engineer', 'job_position_Associate Interactive Developer L2',
'job_position_Associate L1', 'job_position_Associate Principal
Engineer', 'job_position_Associate Processor',
'job_position_Associate Product Lead', 'job_position_Associate
Product Manager', 'job_position_Associate Professional Developer',
'job_position_Associate Professor ', 'job_position_Associate Project
Manager ', 'job_position_Associate Software Developer',
```

```
'job_position_Associate Software Engineer', 'job_position_Associate
Software developer', 'job_position_Associate System Engineer',
'job_position_Associate System Engineer - Trainee',
'job_position_Associate System Engineer-Trainee',
'job_position_Associate Tech Lead', 'job_position_Associate Technical
Engineer ', 'job_position_Associate Technical Lead',
'job_position_Associate Technology', 'job_position_Associate
consultant', 'job_position_Associate consultant - Developer',
'job_position_Associate manager', 'job_position_Associate
professional', 'job_position_Associate software Engineer',
'job_position_Associate software engineer', 'job_position_Associate
system engineer ', 'job_position_Associate technical support',
'job_position_Associate- Futures Engineering Team',
'job_position_Assosiate Product Manager', 'job_position_Assosiate
software engineer', 'job_position_Asst. Manager', 'job_position_Audit
Manager', 'job_position_Auditing', 'job_position_Auditor',
'job_position_Automation Architect', 'job_position_Automation
Developer', 'job_position_Automation Engineer',
'job_position_Automation Expert', 'job_position_Automation Test
Engineer', 'job_position_Automation Test Enginner',
'job_position_Automation developer', 'job_position_Automation
engineer ', 'job_position_Azure data Factory', 'job_position_BA',
'job_position_BDA', 'job_position_BDM', 'job_position_BTA',
'job_position_Back office executive Admin', 'job_position_Backend
Architect', 'job_position_Backend Engineer', 'job_position_Backend
Engineering', 'job_position_Basis administrator',
'job_position_Bengaluru', 'job_position_Big data Developer',
'job_position_Branch Manager', 'job_position_Branch manager',
'job_position_Business Analyst / Product Owner',
'job_position_Business Analyst Lead', 'job_position_Business
Development', 'job_position_Business System Analyst',
'job_position_Business Technology Analyst', 'job_position_Business
analysts ', 'job_position_Business analytics ',
'job_position_Business consultant ', 'job_position_Business
development executive', 'job_position_Business development manager',
'job_position_Business operations new associate',
'job_position_Business process lead', 'job_position_CRC',
'job_position_CTO and Co Founder', 'job_position_Calculator',
"job_position_Can't reveal", 'job_position_Cashier',
'job_position_Catagory manager', 'job_position_Chief People Officer',
'job_position_Chief Technology Officer', 'job_position_City Transport
Lead', 'job_position_Clerk', 'job_position_Cloud ',
'job_position_Cloud Data architect', 'job_position_Cloud Engineer',
'job_position_Cloud Security Analyst', 'job_position_Cloud Software
Engineer ', 'job_position_Cloud Support Associate', 'job_position_Co-
Founder', 'job_position_Co-founder', 'job_position_Co-founderg',
'job_position_Compliance auditor', 'job_position_Compliance operation
', 'job_position_Computer Scientist 1', 'job_position_Consultant',
'job_position_Consultant ', 'job_position_Consultant Development',
'job_position_Content Manager Intern', 'job_position_Coo',
'job_position_Core Tech Engineer / Researcher', 'job_position_Credit
Risk ', 'job_position_Credit risk manager', 'job_position_Customer
Service Executive', 'job_position_Customer Support specialist',
```

```
'job_position_Customer service executive', 'job_position_Cyber
Security Engineer', 'job_position_DATA ASSOCIATE',
'job_position_DEVELOPER ASOCIATE', 'job_position_Data Analyst',
'job_position_Data Eingineer', 'job_position_Data Engineer',
'job_position_Data Engineer 2', 'job_position_Data Engineer III',
'job_position_Data Operations Manager', 'job_position_Data Science
Analyst', 'job_position_Data Scientist', 'job_position_Data Scientist
2', 'job_position_Data Scientist II', 'job_position_Data Specialist',
'job_position_Data Visualization Engineer', 'job_position_Data
Warehouse Developer', 'job_position_Data analayst',
'job_position_Database Administrator', 'job_position_Database
developer', 'job_position_Delivery Project Lead',
'job_position_Deputy Manager', 'job_position_Deputy general manager',
'job_position_Design Engineer', 'job_position_Design specialist',
'job_position_DevOps/Cloud Engineer', 'job_position_Developer &
Production Support', 'job_position_Developer 2',
'job_position_Developer Associate', 'job_position_Devlopment
Engineer', 'job_position_Devops Engineer', 'job_position_Digital
Content Associate', 'job_position_Digital Marketing',
'job_position_Digital marketing manager ', 'job_position_Director
Engineering', 'job_position_Dot NET developer', 'job_position_ENO',
'job_position_ERP Application Developer', 'job_position_ERP
Associate', 'job_position_ERP Consultant', 'job_position_ETL
Developer', 'job_position_Electrical technician',
'job_position_Embedded Software Development Engineer',
'job_position_Embedded Software Engineer', 'job_position_Engeneering
Lead', 'job_position_Engineer', 'job_position_Engineer 1',
'job_position_Engineer 2', 'job_position_Engineer I',
'job_position_Engineer trainee', 'job_position_Engineering Intern',
'job_position_Engineering Intern ', 'job_position_Engineering
Leadership', 'job_position_Engineering Manager II',
'job_position_Engineering Program Manager', 'job_position_Engineering
Team Lead', 'job_position_Engineet', 'job_position_Entrepreneurship
', 'job_position_Escalation Engineer', 'job_position_Exchange
Coordinator', 'job_position_Executive Operation ',
'job_position_Executive Software Developer', 'job_position_F&b
service', 'job_position_Farmer', 'job_position_Fibre Engineer',
'job_position_Finance Lead', 'job_position_Finance and Accounting ',
'job_position_Financial analyst', 'job_position_Firmware Engineer ',
'job_position_Firmware engineer ', 'job_position_Firmware software
engineer l', 'job_position_Flutter App Engineer',
'job_position_Forex', "job_position_Founders' Office",
'job_position_Freelance SDE', 'job_position_Freelancer',
'job_position_Fresher', 'job_position_Fresher student',
'job_position_Front End Dev', 'job_position_Front End Developer',
'job_position_Front end developer', 'job_position_Front end developer
', 'job_position_Front end web developer', 'job_position_Front-End
Developer', 'job_position_Frontend Engineer', 'job_position_Full
Stack Developer Intern', 'job_position_Full Stack Engineer',
'job_position_Full stack web developer', 'job_position_Full-stack web
developer', 'job_position_FullStack Engineer',
'job_position_Fullstack Engineer', 'job_position_Graduate Intern',
'job_position_Graduate Programmer', 'job_position_Graduate Research
```

```
Assistant', 'job_position_Graduate Software Developer',
'job_position_Graduate engineer  trainee', 'job_position_Graduate
engineer trainee', 'job_position_Graphic designer',
'job_position_Growth', 'job_position_Growth Manager',
'job_position_H', 'job_position_HOD', 'job_position_HVAC Algorithm
developer', 'job_position_Hardware engineer', 'job_position_Head
Scientist', 'job_position_Head of Analytics', 'job_position_Human
resource', 'job_position_IT Engineer', 'job_position_IT analyst',
'job_position_IT coordinator', 'job_position_IT mManager',
'job_position_IaaS  Engineer', 'job_position_Implementation
engineer', 'job_position_Info Technology Sr. Associate',
'job_position_Instructional Apprentice', 'job_position_Instructor',
'job_position_Insurance agent ', 'job_position_Integration',
'job_position_Integration Developer ', 'job_position_Integration
Engineer', 'job_position_Intern', 'job_position_Intern - Software
developer', 'job_position_Intern-data analyst',
'job_position_International Coding Instructor',
'job_position_Internship', 'job_position_IoT Consultant',
'job_position_It executive', 'job_position_It trainee',
'job_position_Java Back End Developer', 'job_position_Java Software
Engineer', 'job_position_Java devloper', 'job_position_Java
microservices developer', 'job_position_Jr. Prod Specialist',
'job_position_Jr. Software Engineer', 'job_position_Junior Associate,
Technology', 'job_position_Junior Front End Eng',
'job_position_Junior Software developer', 'job_position_Junior
consultant', 'job_position_Junior developer', 'job_position_Junior
software developer', 'job_position_Kam', 'job_position_Key account
manager', 'job_position_L4', 'job_position_Lead', 'job_position_Lead
- MIS', 'job_position_Lead Analyst', 'job_position_Lead Android
Developer', 'job_position_Lead Consultant', 'job_position_Lead
Consultant Technology', 'job_position_Lead Design Engineer',
'job_position_Lead Engineer - Software Development',
'job_position_Lead Engineer II', 'job_position_Lead Software
Developer', 'job_position_Lead Software Development Engineer',
'job_position_Lead Software Engineer', 'job_position_Lead Software
Engineer in Test', 'job_position_Lead UI Engineer',
'job_position_Lead software Engineer', 'job_position_Lead software
engineer', 'job_position_Learning web designing',
'job_position_Lecturer', 'job_position_Leisure Manager',
'job_position_Linux System Administrator', 'job_position_Linux
administrator', 'job_position_Loan processing',
'job_position_Logistics supply chain', 'job_position_MEAN Stack
Developer', 'job_position_ML Engineer', 'job_position_MTS',
'job_position_MTS - II', 'job_position_MTS 3', 'job_position_MTS
Intern', 'job_position_MTS-2', 'job_position_MTS-3',
'job_position_MTS2', 'job_position_Machine Learning Data Associate ',
'job_position_Machine Learning Developer', 'job_position_Machine
Learning Engineer', 'job_position_Machine Learning Engineer Intern',
'job_position_Machine Learning Intern', 'job_position_Machine
learning engineer', 'job_position_Mainframe Developer',
'job_position_Mainframe developer', 'job_position_Maintenance
Engineer ', 'job_position_Management executive',
'job_position_Manager', 'job_position_Manager ',
```

```
'job_position_Manager candidate', 'job_position_Manager-CX',
'job_position_Manual Tester', 'job_position_Maricketing',
'job_position_Marketing Professional ', 'job_position_Matlab
programmer', 'job_position_Mechanical engineering ',
'job_position_Mechanical engineers', 'job_position_Medical field',
'job_position_Member Of Technical Staff', 'job_position_Member Of
Technical Staff 2', 'job_position_Member Technical',
'job_position_Member Technical Staff', 'job_position_Member Technical
Staff -1', 'job_position_Member Technical Staff III',
'job_position_Member Technical Staff Zoho', 'job_position_Member of
Technical Staff', 'job_position_Member of Technical Staff (JAVA)',
'job_position_Member of Technical Staff (MTS)', 'job_position_Member
of Technical Staff - 2', 'job_position_Member of Technical Staff 2',
'job_position_Member of Technical Staff 3', 'job_position_Member of
Technical Staff 4', 'job_position_Member of Technical Staff Level 2',
'job_position_Member of Technical Staff at Nineleaps',
'job_position_Member of technical staff', 'job_position_Member
technial Staff', 'job_position_Member technical staff 2',
'job_position_Messenger come driver', 'job_position_Middleware
Engineer', 'job_position_Mobile Application Developer',
'job_position_Mobile Application Engineer(Android & iOS)',
'job_position_Module Lead', 'job_position_Module Lead Software
Engineer', 'job_position_Mulesoft developer', 'job_position_N',
'job_position_NCE', 'job_position_Na', 'job_position_Network
Administrator ', 'job_position_Network Software Engineer',
'job_position_Network administrator', 'job_position_Networking',
'job_position_New graduate', 'job_position_No', 'job_position_No
job', 'job_position_Non Coder', 'job_position_Not applicable',
'job_position_Not employed', 'job_position_Now iam not working,
waiting for job', 'job_position_Null', 'job_position_Office
management', 'job_position_Online Reputation Manager',
'job_position_Operation Executive', 'job_position_Operation
associate', 'job_position_Operations Expert',
'job_position_Operations Lead', 'job_position_Operations Manager',
'job_position_Operations Manager ', 'job_position_Operations
analyst', 'job_position_Oracle Apps Technical Consultant',
'job_position_Oracle DBA', 'job_position_Oracle Lead DBA',
'job_position_Other', 'job_position_Other ', 'job_position_Others',
'job_position_Owner', 'job_position_PA', 'job_position_PAT',
'job_position_PDP EXECUTIVE', 'job_position_PGT Computer Science',
'job_position_PMO', 'job_position_Package IImplementation Associate',
'job_position_Performance Engineer', 'job_position_Personal Banker',
'job_position_Platform Engineer', 'job_position_Pop',
'job_position_Pop engineer ', 'job_position_Power plant engineer',
'job_position_Power platform Developer ', 'job_position_Pragrammer
Analyst', 'job_position_Principal Application Engineer',
'job_position_Principal Engineer - 1', 'job_position_Principal
Product Engineer', 'job_position_Principal Software Engineer',
'job_position_Principal Solution Architect', 'job_position_Principal
consultant', 'job_position_Principle Member of Technical Staff',
'job_position_Process Executive ', 'job_position_Process associate',
'job_position_Product Designer', 'job_position_Product Develloper',
'job_position_Product Developer', 'job_position_Product Development
```

```
Engineer', 'job_position_Product Engineer (Backend)',
'job_position_Product Manager', 'job_position_Product Specialist',
'job_position_Product development engineer', 'job_position_Product
solution Engineer', 'job_position_Production Engineering Senior
Analyst', 'job_position_Professional ', 'job_position_Professional
Application Delivery I', 'job_position_Professional Services
Engineer', 'job_position_Professor', 'job_position_Program Manager',
'job_position_Program Manager Intern', 'job_position_Programmar
analyst', 'job_position_Programmer Analyst', 'job_position_Programmer
Analyst ', 'job_position_Programmer Analyst 1',
'job_position_Programmer Analyst 2', 'job_position_Programmer Analyst
Trainee', 'job_position_Programmer analyst', 'job_position_Project
Associate', 'job_position_Project Engineer', 'job_position_Project
Lead', 'job_position_Project Leader', 'job_position_Project Officer
', 'job_position_Project Trainee', 'job_position_Project engineer',
'job_position_Project management profession', 'job_position_Pune',
'job_position_Python Devloper', 'job_position_Q', 'job_position_QA
Engineer', 'job_position_QA intern', 'job_position_QA/QC Engineer',
'job_position_QAE-Intern', 'job_position_Quality Analyst',
'job_position_Quality Analyst ', 'job_position_Quality Associate',
'job_position_Quality Assurance Manager', 'job_position_Quality
Engineer', 'job_position_Quality Enginner',
'job_position_Quantitative Analyst', 'job_position_R & D Engineer-2',
'job_position_R & D engineer', 'job_position_R&D Engineer',
'job_position_RD Engineer II', 'job_position_RD Enginner II',
'job_position_RETAIL ASSCOCIATE', 'job_position_Radio frequency
Engineer', 'job_position_React Native developer',
'job_position_Reasearch Engineer', 'job_position_Recovery officer',
'job_position_Relationship officer', 'job_position_Release Engineer',
'job_position_Reseach Analyst', 'job_position_Research Assistant',
'job_position_Research Assistant / Software Engineer',
'job_position_Research Engineer', 'job_position_Research Engineer 2',
'job_position_Research Engineers', 'job_position_Research Intern -
Machine Learning', 'job_position_Research analyst',
'job_position_Researcher', 'job_position_Risk Analyst',
'job_position_Risk Investigator', 'job_position_Risk model
developer', 'job_position_S/w Engineer 2', 'job_position_SAP ABAP
Developer', 'job_position_SAP C4C Functional Consultant',
'job_position_SAP CPQ Consultant', 'job_position_SAP Technical
consultant', 'job_position_SDE - 3 ', 'job_position_SDE - II',
'job_position_SDE 1', 'job_position_SDE 1 ', 'job_position_SDE 2',
'job_position_SDE 3', 'job_position_SDE 4', 'job_position_SDE II',
'job_position_SDE Intern', 'job_position_SDE, Lead',
'job_position_SDE-1', 'job_position_SDE-2', 'job_position_SDE-3',
'job_position_SDE-I', 'job_position_SDE-II', 'job_position_SDE2',
'job_position_SDE3', 'job_position_SDEII', 'job_position_SDET',
'job_position_SDET - Intern', 'job_position_SDET 3',
'job_position_SDET senior', 'job_position_SDET-2', 'job_position_SE
II', 'job_position_SE(R&D)', 'job_position_SENIOR MANAGER',
'job_position_SMTS', 'job_position_SOFTWARE ENG',
'job_position_SOFTWARE ENGINEER III', 'job_position_SOftware Engineer
I', 'job_position_SQL, PLSQL Developer', 'job_position_SRE',
'job_position_Sales', 'job_position_Sales ', 'job_position_Sales and
```

```
collection executive ', 'job_position_Sales manager',
'job_position_Sales professional', 'job_position_Salesforce
Consultant', 'job_position_Salesforce Developer', 'job_position_Sap
Abap Developer ', 'job_position_Scientist - C',
'job_position_Secretary', 'job_position_Security Delivery
Specialist', 'job_position_Security Leadership',
'job_position_Security Researcher ', 'job_position_Security analyst
', 'job_position_Self job', 'job_position_Seller Support Associate',
'job_position_Seller support', 'job_position_Senior  Software
Engineer', 'job_position_Senior Administrator', 'job_position_Senior
Analyst', 'job_position_Senior Analyst Programmer',
'job_position_Senior Analysts', 'job_position_Senior Android
Developer', 'job_position_Senior Application Engineer',
'job_position_Senior Applications Engineer', 'job_position_Senior
Associate', 'job_position_Senior Associate ', 'job_position_Senior
Associate - (Index Technology - Production Support)',
'job_position_Senior Associate Platform L1', 'job_position_Senior
Associate Technology Level1', 'job_position_Senior Business Analyst',
'job_position_Senior Business Operations Analyst',
'job_position_Senior Chat Associate', 'job_position_Senior
Consultant', 'job_position_Senior Data Engineer',
'job_position_Senior Data Scientist', 'job_position_Senior Database
Engineer', 'job_position_Senior Developer', 'job_position_Senior
Development Engineer', 'job_position_Senior Engineer',
'job_position_Senior Engineer Software', 'job_position_Senior
Enginner', 'job_position_Senior Executive', 'job_position_Senior
Front End Engineer', 'job_position_Senior Frontend Developer',
'job_position_Senior Frontend Engineer', 'job_position_Senior Lead
Analyst', 'job_position_Senior Member Of Technical Staff',
'job_position_Senior Member Of Technical Staff - R&D - Machine
Learning', 'job_position_Senior Member Technical Staff',
'job_position_Senior Member of Technical Staff', 'job_position_Senior
Mobile Applications Developer (Android/iOS)', 'job_position_Senior
Mobile Developer', 'job_position_Senior Process Executive',
'job_position_Senior Product Development Engineer (Senior
Developer)', 'job_position_Senior QA Engineer', 'job_position_Senior
Quantitative Analyst', 'job_position_Senior Research Associate',
'job_position_Senior Software  Development Engineer',
'job_position_Senior Software Consultant', 'job_position_Senior
Software Developer', 'job_position_Senior Software Development
Enggineer', 'job_position_Senior Software Development Engineer',
'job_position_Senior Software Development Engineer (Backend)',
'job_position_Senior Software Development Engineer II',
'job_position_Senior Software Development Engineer in Test',
'job_position_Senior Software Engineer (.Net Backend)',
'job_position_Senior Software Engineer (Backend)',
'job_position_Senior Software Engineer (front end)',
'job_position_Senior Software Engineer - Infrastructure',
'job_position_Senior Software Engineer 1', 'job_position_Senior
Software Engineer lead', 'job_position_Senior Software Engineer, QA',
'job_position_Senior Software Engineering', 'job_position_Senior
Software engineeer', 'job_position_Senior Sofware Engineer',
'job_position_Senior Solution Engineer ', 'job_position_Senior Staff
```

```
Software Engineer', 'job_position_Senior System Engineer ',
'job_position_Senior System Specialist', 'job_position_Senior Systems
Engineer', 'job_position_Senior Systems Engineer ',
'job_position_Senior Technical Assistant', 'job_position_Senior
Technical Specialist ', 'job_position_Senior UI Enginner',
'job_position_Senior Web Developer', 'job_position_Senior WebMethods
Developer', 'job_position_Senior administrator ',
'job_position_Senior associate', 'job_position_Senior consulant',
'job_position_Senior consultant', 'job_position_Senior data
engineer', 'job_position_Senior developer', 'job_position_Senior
devops', 'job_position_Senior engineer', 'job_position_Senior project
engineer', 'job_position_Senior software Test Engineer',
'job_position_Senior software analyst', 'job_position_Senior software
developer', 'job_position_Senior system engineer',
'job_position_Senior systems Engineer ', 'job_position_Senior systems
engineer', 'job_position_Senior test engineer',
'job_position_SeniorSoftwareEngineer', 'job_position_Server
Engineer', 'job_position_Service Delivery Specialist',
'job_position_ServiceNow Developer', 'job_position_Services IT
Development Program Senior Associate ', 'job_position_SharePoint
Developer', 'job_position_Shift controller', 'job_position_Site
Reliability Engineer', 'job_position_Site Reliability Engineer ',
'job_position_Snr Analyst Programmer/Java Developer',
'job_position_So', 'job_position_Soft skill Trainer at Skill India
Government project', 'job_position_Software', 'job_position_Software
Engineer', 'job_position_Software / web developer',
'job_position_Software Analyst', 'job_position_Software Associate',
'job_position_Software Consultant', 'job_position_Software Dev
Engineer 1', 'job_position_Software Dev. Engineer',
'job_position_Software Developer (SDE-3)', 'job_position_Software
Developer - Android', 'job_position_Software Developer - PL10',
'job_position_Software Developer - iOS', 'job_position_Software
Developer 1', 'job_position_Software Developer 2',
'job_position_Software Developer Consultant', 'job_position_Software
Developer I', 'job_position_Software Developer II',
'job_position_Software Developer Intern', 'job_position_Software
Developer grade-1', 'job_position_Software Development Analyst',
'job_position_Software Development Consultant 1',
'job_position_Software Development Engineer - 2',
'job_position_Software Development Engineer - I',
'job_position_Software Development Engineer - II',
'job_position_Software Development Engineer - III',
'job_position_Software Development Engineer - Intern',
'job_position_Software Development Engineer -2 ',
'job_position_Software Development Engineer 1',
'job_position_Software Development Engineer 2',
'job_position_Software Development Engineer 2 ',
'job_position_Software Development Engineer Intern',
'job_position_Software Development Engineer in Test',
'job_position_Software Development Engineering Intern',
'job_position_Software Development Enginner', 'job_position_Software
Development Intern', 'job_position_Software Development Manager',
'job_position_Software Devloper', 'job_position_Software Eng',
```

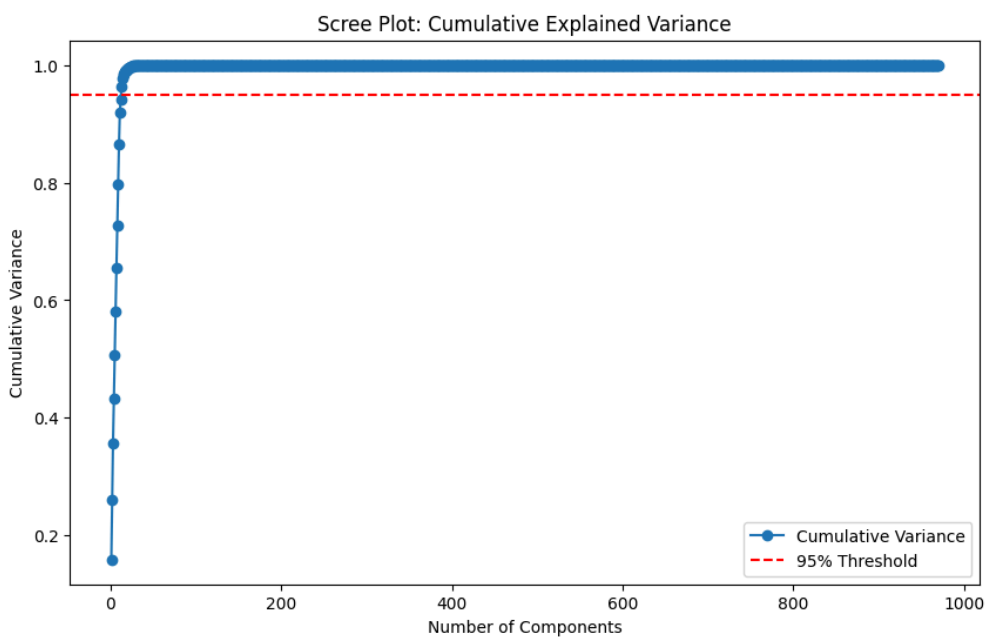
```
'job_position_Software Engineer  - Programmer Analyst',
'job_position_Software Engineer (Android)', 'job_position_Software
Engineer (Backend)', 'job_position_Software Engineer (Full Stack)',
'job_position_Software Engineer (Full stack)', 'job_position_Software
Engineer (Intern)', 'job_position_Software Engineer - Frontend',
'job_position_Software Engineer - I', 'job_position_Software Engineer
- II', 'job_position_Software Engineer - Operations',
'job_position_Software Engineer - Testing', 'job_position_Software
Engineer 1', 'job_position_Software Engineer 2',
'job_position_Software Engineer 2 (Backend)', 'job_position_Software
Engineer 2 (Full Stack)', 'job_position_Software Engineer Android',
'job_position_Software Engineer Android Intern',
'job_position_Software Engineer Associate', 'job_position_Software
Engineer I', 'job_position_Software Engineer II',
'job_position_Software Engineer III', 'job_position_Software Engineer
IV', 'job_position_Software Engineer Intern', 'job_position_Software
Engineer L2', 'job_position_Software Engineer R&D',
'job_position_Software Engineer Trainee(ML)', 'job_position_Software
Engineer in Test', 'job_position_Software Engineer(Advanced
Analytics)', 'job_position_Software Engineer(R&D) Intern ',
'job_position_Software Engineer, Associate', 'job_position_Software
Engineering', 'job_position_Software Engineering - Engineer 2',
'job_position_Software Engineering Analyst', 'job_position_Software
Engineering Contractor', 'job_position_Software Engineering Intern',
'job_position_Software Engineering Lead', 'job_position_Software
Enginner', 'job_position_Software Enginnering Specialist',
'job_position_Software Prod & Plat Eng Analyst',
'job_position_Software QA Engineer', 'job_position_Software Test
Engineer', 'job_position_Software consultant', 'job_position_Software
developer (Data engineer)', 'job_position_Software developer
(automation)', 'job_position_Software developer/Application
Developer', 'job_position_Software development ',
'job_position_Software development engineer 1 ',
'job_position_Software development engineer 2',
'job_position_Software development specialist',
'job_position_Software engineer - 2', 'job_position_Software engineer
3', 'job_position_Software engineer III ( sde2) ',
'job_position_Software engineer analyst ', 'job_position_Software
qa', 'job_position_Softwear engineer', 'job_position_Solution
Architect - Apprentice', 'job_position_Solution Engineer',
'job_position_Solution Integration Consultant I',
'job_position_Solutions Architect', 'job_position_Solutions
Designer', 'job_position_Solutions Specialist', "job_position_Some
data entry operator like some copy's write.type and upload",
'job_position_Some random title', 'job_position_Specialist',
'job_position_Specialist Programmer', 'job_position_Specialist
Programmer (PP)', 'job_position_Sps Associate', 'job_position_Sr Lead
Engineer', 'job_position_Sr Software Engg - Android',
'job_position_Sr Software Engineer', 'job_position_Sr consultant',
'job_position_Sr representative ', 'job_position_Sr software engg',
'job_position_Sr software engineer ', 'job_position_Sr. Associate
PLatform L2', 'job_position_Sr. EXECUTIVE ', 'job_position_Sr. MIS
Executive', 'job_position_Sr. Operation Executive', 'job_position_Sr.
```

```
Software Engineer', 'job_position_Sr. Software Engineer(Android)',
'job_position_Sr. Technical Architect', 'job_position_Sr. iOS
Engineer', 'job_position_Sr. iOS developer', 'job_position_Sr.
technical manager', 'job_position_Staff Consultant',
'job_position_Staff Developer', 'job_position_Staff Software
Engineer', 'job_position_Storw', 'job_position_Strategy and new
initiatives', 'job_position_Student', 'job_position_Student In
Computer Application', 'job_position_Student Intern',
'job_position_Sub-Inspector', 'job_position_Subject Matter Expert',
'job_position_Summer Technology Analyst (Software Developer Intern)',
'job_position_Support', 'job_position_Support Engineer',
'job_position_System Engineer', 'job_position_System
Engineer,FullStack Developer', 'job_position_System Software
engineer', 'job_position_System admin', 'job_position_System
engineer', 'job_position_System engineer ', 'job_position_System
software Engineer', 'job_position_Systems Analyst',
'job_position_Systems Operations Engineer', 'job_position_TA',
'job_position_TX SME', 'job_position_Talent Acquisition SSpecialist',
'job_position_Talent Acquisition Specialist', 'job_position_Teacher',
'job_position_Teacher ', 'job_position_Teaching',
'job_position_Teaching Assistant', 'job_position_Teaching Assistant
', 'job_position_Teaching field', 'job_position_Team Lead',
'job_position_Team Lead - Android', 'job_position_Team Lead Mobile',
'job_position_Team Manager', 'job_position_Team lead',
'job_position_Teammate', 'job_position_Tecchnical lead',
'job_position_Tech Lead Software Engineer', 'job_position_Tech
Manager', 'job_position_Tech Person', 'job_position_Tech
architecture', 'job_position_Tech lead /Team Lead ',
'job_position_TechOps Engineer 1', 'job_position_Techinical Lead',
'job_position_Technical  consultant', 'job_position_Technical
Associate', 'job_position_Technical Engineer ',
'job_position_Technical Leader', 'job_position_Technical Manager ',
'job_position_Technical Mentor', 'job_position_Technical Program
Program', 'job_position_Technical Support Engineer',
'job_position_Technical Test Lead', 'job_position_Technical
consultant', 'job_position_Technical consulting ',
'job_position_Technical head', 'job_position_Technical leader',
'job_position_Technical officer', 'job_position_Technical support
Engineer', 'job_position_Technical support engineer ',
'job_position_Technician', 'job_position_Technology Analyst',
'job_position_Technology Analyst II', 'job_position_Technology
Associate', 'job_position_Technology Consultant',
'job_position_Technology Specialist ', 'job_position_Technology
analyst', 'job_position_Test Analyst', 'job_position_Test Automation
Engineer', 'job_position_Test Technician, Specialist',
'job_position_Tester', 'job_position_Third party provider',
'job_position_Tool room Engineer, Mechanical Engineer ',
'job_position_Trainee', 'job_position_Trainee Decision Scientist',
'job_position_Trainee Engineer (Java Developer)',
'job_position_Trainee(Intern)', 'job_position_Trainer',
'job_position_Trainer ', 'job_position_Transaction Risk Investigator
', 'job_position_Tsc', 'job_position_Tuition teacher',
'job_position_Typing', 'job_position_UI Developer Intern',
```

```
'job_position_Ucm', 'job_position_Underwriting Officer',
'job_position_Unempolyed', 'job_position_User Experience Specialist',
'job_position_Vendor management', 'job_position_Vice President',
'job_position_Vice president', 'job_position_Voice President',
'job_position_Web / UI Designer', 'job_position_Web Application
Developer', 'job_position_Web Designer', 'job_position_Web
Designer/Developer Intern', 'job_position_Web Developer (Full
Stack)', 'job_position_Web engineer', 'job_position_Webcenter
consultant', 'job_position_WordPress developer',
'job_position_Worker', 'job_position_X', 'job_position_Young
Professional II', 'job_position_Zomato', 'job_position_Zomato ',
'job_position_air force', 'job_position_application development team
lead', 'job_position_associate software engineer',
'job_position_ayS', 'job_position_consultanat',
'job_position_consultant', 'job_position_decision analytics
associate', 'job_position_embedded software engineer',
'job_position_employee', 'job_position_entrepreneur',
'job_position_freelance', 'job_position_front end engineer',
'job_position_full stack devloper', 'job_position_highway engineer',
'job_position_iOS / Swift Developer', 'job_position_iOS Application
Developer', 'job_position_iOS Engineer', 'job_position_iOS Software
Developer', 'job_position_junior web developer', 'job_position_linux
administrator ', 'job_position_member of technical staff',
'job_position_na', 'job_position_network function virtualisation',
'job_position_no', 'job_position_no job', 'job_position_operation',
'job_position_platform engineer', 'job_position_process associate',
'job_position_program analyst trainee', 'job_position_programmer
analyst', 'job_position_project assistant ', 'job_position_project
engineer', 'job_position_senior Consultant', 'job_position_senior
engineer', 'job_position_senior software engineer-L2',
'job_position_software developer - UI', 'job_position_software
engineer 1', 'job_position_software engineer 2B', 'job_position_sr.
developer', 'job_position_student', 'job_position_support escalation
engineer', 'job_position_system engineer', 'job_position_system
software engineer', 'job_position_technology analyst']
```

In [121]:
```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Apply PCA
pca = PCA()
pca.fit(df)

# Explained variance ratio
explained_variance = pca.explained_variance_ratio_
cumulative_variance = explained_variance.cumsum()

# Determine number of components for ~95% variance
n_components_95 = (cumulative_variance < 0.95).sum() + 1
print(f"Number of components to retain ~95% variance: {n_components_95}")

# Transform data
df_pca = pca.transform(df)[:, :n_components_95]

# Plot Scree Plot
plt.figure(figsize=(10,6))
plt.plot(range(1, len(cumulative_variance)+1), cumulative_variance,
marker='o', label='Cumulative Variance')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% Threshold')
plt.title("Scree Plot: Cumulative Explained Variance")
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Variance")
plt.legend()
plt.show()
```

```
Number of components to retain ~95% variance: 13
```



Key Insights

1. Dimensionality Reduction

- Out of 970 original features, only 13 principal components are needed to capture ~95% of the variance.
- This is a massive reduction (from 970 → 13), which means:

- Most of the original features are highly redundant or correlated.
- PCA has effectively compressed the information into a very small set of components.

2. Variance Distribution

- The curve rises sharply in the first few components and then flattens almost immediately.
- First few components (1–5) explain the majority of the variance.
- After ~13 components, additional components add negligible variance (almost flat line).

3. Implications

- High multicollinearity in original features (especially after One-Hot Encoding).
- Using PCA-transformed features will:

- Improve computational efficiency.
- Reduce risk of overfitting.
- Make models more stable.

In [122]:
```
df_pca
```

Out[122]:
```
array([[-1.14620934, -1.57385275,  0.26990534, ..., -1.67480803,
         0.09453682, -0.12682908],
       [-0.40882192,  0.4788812 , -0.08233408, ...,  0.28231117,
         0.66051215, -0.25214152],
       [ 1.47279452, -0.59992925, -1.37770048, ...,  0.15296815,
         0.27755371,  0.38285419],
       ...,
       [-0.1137724 ,  0.62075922, -0.16586061, ...,  0.32838559,
         0.59261874,  0.67323355],
       [-0.00639212,  0.58957338, -0.25910553, ...,  0.1458669 ,
         0.61201657,  0.6624202 ],
       [ 1.29161435, -0.68506632, -1.22553052, ...,  0.86454528,
         0.49037586,  0.54795008]])
```

Model building

In [123]:

```
## Data Splitting

from sklearn.model_selection import train_test_split

# Perform 80-20 split
train_data, test_data = train_test_split(df_pca, test_size=0.2,
random_state=42)

print(f"Total samples: {len(df_pca)}")
print(f"Train set size: {len(train_data)}")
print(f"Test set size: {len(test_data)}")
```

```
Total samples: 192683
Train set size: 154146
Test set size: 38537
```

Checking Clustering Tendency

In [124]:
```
from sklearn.neighbors import NearestNeighbors
import random

def hopkins_statistic(X, sample_size=None):
    if sample_size is None:
        sample_size = int(0.1 * X.shape[0])  # 10% of data
    X = np.array(X)

    # Random sample from data
    sample_indices = random.sample(range(X.shape[0]), sample_size)
    sample = X[sample_indices]

    # Random points in feature space
    random_points = np.random.uniform(np.min(X, axis=0), np.max(X, axis=0),
(sample_size, X.shape[1]))

    # Nearest neighbors
    nbrs = NearestNeighbors(n_neighbors=1).fit(X)
    u_dist, _ = nbrs.kneighbors(random_points)
    w_dist, _ = nbrs.kneighbors(sample)

    hopkins = np.sum(u_dist) / (np.sum(u_dist) + np.sum(w_dist))
    return hopkins

hopkins_value = hopkins_statistic(df_pca)
print(f"Hopkins Statistic: {hopkins_value}")
```

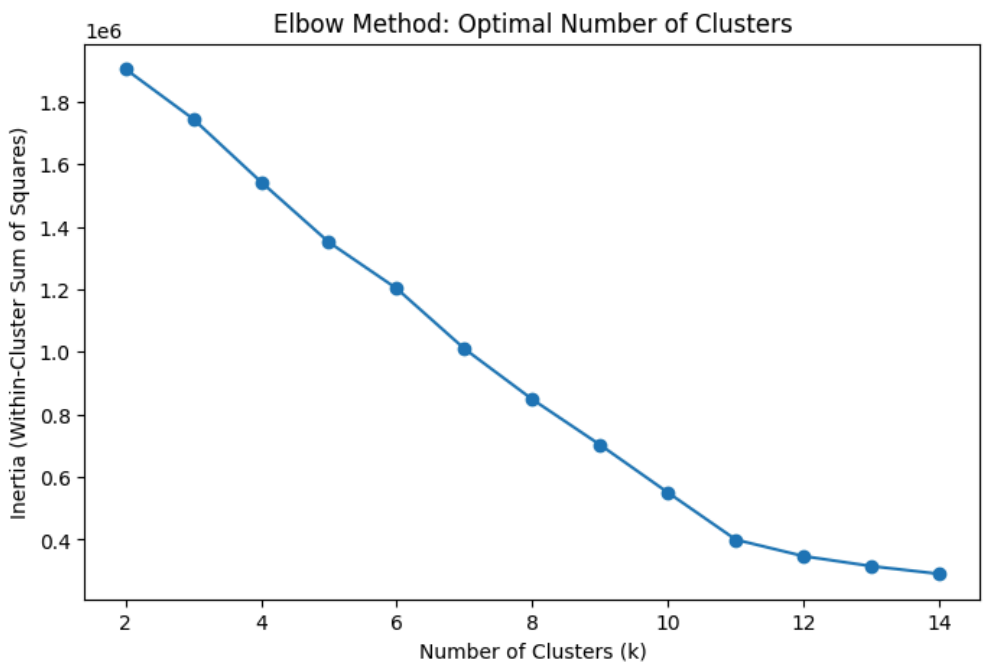
```
Hopkins Statistic: 1.0
```

Key Insights

Extremely High Clustering Tendency :
- Hopkins value of 1.0 indicates the data is highly non-random and has a very strong clustering structure.
- This is the maximum possible value, meaning the points are far from uniformly distributed.

In [125]:
```
## Selecting Optimal Number of Clusters
```

In [126]:
```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Range of clusters to test
k_values = range(2, 15)
inertia_values = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(train_data)
    inertia_values.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(k_values, inertia_values, marker='o')
plt.title("Elbow Method: Optimal Number of Clusters")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia (Within-Cluster Sum of Squares)")
plt.show()
```



Key Insights:

- The curve starts steep at k = 2 and gradually flattens.
- The biggest change in slope happens between k = 10 and k = 12, where the inertia reduction becomes minimal.
- After k = 11, adding more clusters gives very little improvement.

- Choosing too many clusters (e.g., >10) would overfit and reduce interpretability without much gain.

Recommendation: Combine Elbow Method with Silhouette Score:

- Silhouette Score often peaks at a lower k than the elbow.
- This helps avoid overfitting.

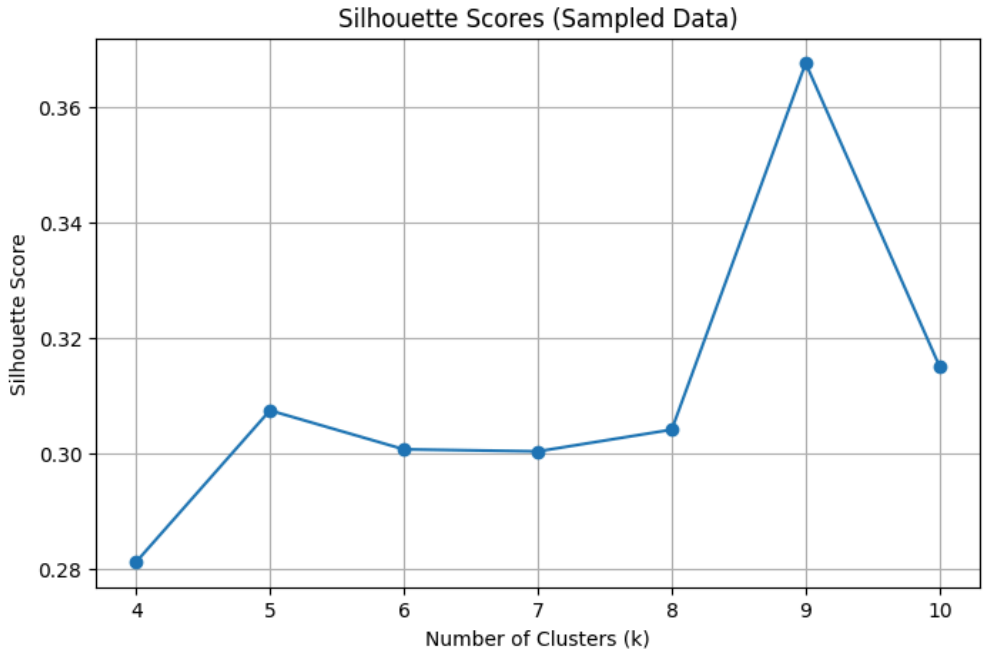
In [127]:
```
train_data
```

Out[127]:
```
array([[-0.55044701,  0.9822062 ,  0.2098284 , ...,  0.36453148,
         0.21923279,  0.52658502],
       [-1.09060664, -1.65681231,  0.2572804 , ..., -1.51566939,
        -0.1604573 , -0.24592337],
       [ 0.4080305 , -0.09531233, -0.712762  , ...,  0.01999126,
         0.48749149, -0.41818   ],
       ...,
       [-3.058724  , -4.51435856,  2.18382184, ...,  3.23826453,
        -0.48937875, -0.13835502],
       [ 0.28811972,  0.26194128, -0.45858422, ...,  0.45372755,
         0.43912154,  0.57460407],
       [ 1.56186101, -0.79780371, -1.22924746, ...,  0.10608976,
        -0.17159069, -0.34465135]])
```

Silhouette Score

In [128]:

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Convert NumPy array to DataFrame
df_train = pd.DataFrame(train_data)

# Sample data for faster computation
sample_size = min(10000, len(df_train))  # Use up to 10k rows
df_sample = df_train.sample(n=sample_size, random_state=42)

# Range of clusters to test
k_values = range(4, 11)
silhouette_scores = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(df_sample)
    score = silhouette_score(df_sample, cluster_labels)
    silhouette_scores.append(score)

# Plot Silhouette Scores
plt.figure(figsize=(8, 5))
plt.plot(k_values, silhouette_scores, marker='o')
plt.title("Silhouette Scores (Sampled Data)")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Silhouette Score")
plt.grid(True)
plt.show()

# Print scores
print("Silhouette Scores:")
for k, score in zip(k_values, silhouette_scores):
    print(f"k={k}: {score:.4f}")
```




```
Silhouette Scores:
k=4: 0.2812
k=5: 0.3075
k=6: 0.3008
k=7: 0.3004
k=8: 0.3042
k=9: 0.3676
k=10: 0.3151
```

Key Insights

1. Best Performing Cluster Size

- The highest Silhouette Score is at k = 9 with a score of 0.3676.
- This indicates that 9 clusters provide the best separation and cohesion among clusters in the sampled data.

2. Overall Trend

- Scores start low at k = 4 (0.2812) and gradually improve.
- There is a noticeable jump between k = 8 (0.3042) and k = 9 (0.3676).
- After k = 9, the score drops at k = 10 (0.3151), suggesting that adding more clusters beyond 9 reduces quality.

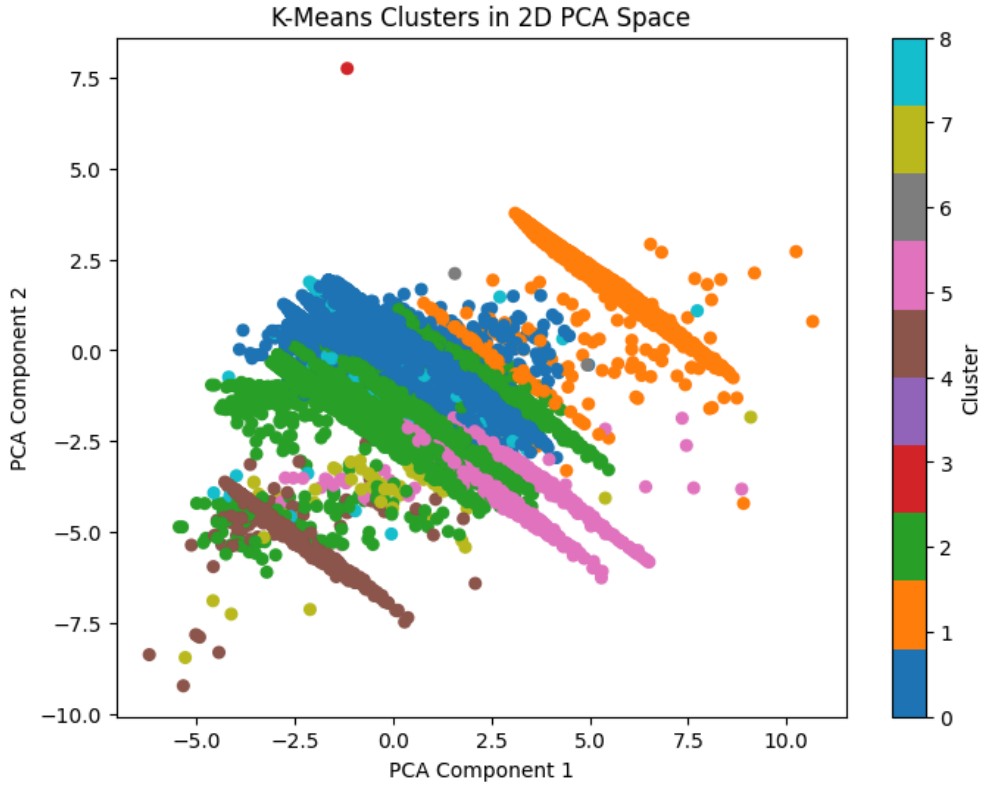
3. Interpretation

- k = 9 seems optimal based on Silhouette Score.
- However, k = 5–6 also have decent scores (~0.30), which might be preferable if interpretability and simplicity are priorities.
- Combining this with the Elbow Method (which suggested k ≈ 11) shows that k = 9 is a good compromise between statistical quality and complexity.

K-means Clustering

In [129]:

```
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Convert train_data and test_data to DataFrames
df_train = pd.DataFrame(train_data)
df_test = pd.DataFrame(test_data)

# Apply KMeans with optimal k
optimal_k = 9
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
kmeans.fit(df_train)

# Assign clusters
df_train['Cluster'] = kmeans.labels_
df_test['Cluster'] = kmeans.predict(df_test)

# Visualize clusters in 2D PCA space
pca_2d = PCA(n_components=2)
train_pca_2d = pca_2d.fit_transform(df_train.drop('Cluster', axis=1))

plt.figure(figsize=(8, 6))
plt.scatter(train_pca_2d[:, 0], train_pca_2d[:, 1], c=df_train['Cluster'],
cmap='tab10', s=30)
plt.title("K-Means Clusters in 2D PCA Space")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.colorbar(label='Cluster')
plt.show()

# Generate cluster profiles (mean values per cluster)
cluster_profiles = df_train.groupby('Cluster').mean()
print("Cluster Profiles (Mean values per cluster):")
print(cluster_profiles)
```




```
Cluster Profiles (Mean values per cluster):
                0         1         2         3          4          5
\
Cluster
0        0.010463  0.359500 -0.324948 -0.159913   0.023031   0.072074
1        5.755469  1.478340  3.304101  0.719077  -1.789126  -1.730612
2       -0.870586 -1.350099  0.616778  0.125862   0.018613   0.013270
3       -0.912212  2.950834  0.546928  5.459764   4.165641  -4.962582
4       -2.852480 -4.764707  1.991214 -0.510871  -2.574585  -3.918000
5        2.833212 -3.748390  5.963357 -0.301470   6.473159   4.345913
6        2.546622  0.305612 -0.452663  5.653991   3.522990  -4.887963
7       -0.472094 -3.917685  4.942470  2.263607 -24.387635  28.761007
8       -0.380594  0.353067 -1.634964  6.791781   1.541375  -0.152090

                  6           7          8          9        10
11  \
Cluster
0          0.002902   -0.000399   0.045629  -0.053817  0.240248
0.007287
1         -0.064347    0.008828  -0.082121   0.269468 -0.981126
0.426492
2          0.005925    0.000904   0.104447  -0.348680 -1.428109
-0.116922
3        252.709696  358.654852  -1.823307  10.111492 -1.433221
-0.318947
4         -0.063453    0.003958  -0.855301   1.924472  3.425530
-0.063385
5         -0.013621    0.008865   0.312127   0.741372  2.722354
0.511195
6        253.318321 -178.941783  -0.660202   8.859785 -1.059178
```

```
0.095337
7          1.010048   -0.007543 -29.906514  -6.508250  3.273539
0.149395
8         -0.348219   -0.015400  -2.077231   4.228214 -0.386357
-0.089436

               12
Cluster
0       -0.025574
1       -0.588675
2        0.244550
3       -0.335827
4        0.061336
5       -0.749159
6       -0.216296
7       -0.237195
8        0.168691
```

Key Insights

1. Cluster Separation (Visualization)

- The PCA 2D plot shows 9 distinct clusters, each represented by a different color.
- Clusters are generally well-separated, confirming strong clustering tendency (Hopkins = 1.0).
- Some clusters (e.g., orange and pink) are elongated, suggesting internal variance, while others (e.g., brown and green) are more compact.
- There is slight overlap between certain clusters, indicating possible similarity in feature space.

2. Cluster Profiles (Mean Feature Values)

- Each cluster has unique mean values across PCA components, indicating distinct patterns.
- Cluster 3 and Cluster 6 stand out with extremely high values in certain dimensions (e.g., component 6 and 10), suggesting these clusters represent outlier or specialized groups.
- Cluster 0 and Cluster 8 have relatively small mean values across all components, indicating they might represent more “average” or balanced groups.
- Cluster 5 shows negative values in many components, suggesting a contrasting profile compared to clusters with positive means.

3. Interpretation

- Clusters are not only visually distinct but also numerically different in their profiles.
- Some clusters (like 3 and 6) may correspond to niche segments or anomalies.
- Balanced clusters (like 0 and 8) could represent the majority population.

Hierarchical Clustering

In [130]:
```
# import pandas as pd
# from sklearn.cluster import AgglomerativeClustering
# from scipy.cluster.hierarchy import dendrogram, linkage
# import matplotlib.pyplot as plt

# # Convert train_data to DataFrame
# df_train = pd.DataFrame(train_data)
# df_train.columns = df_train.columns.astype(str)

# # Sample for dendrogram (reduce to 200 rows)
# sample_df = df_train.sample(n=200, random_state=42)

# # Compute linkage on sample
# linked = linkage(sample_df, method='ward')

# # Plot dendrogram
# plt.figure(figsize=(12, 6))
# dendrogram(linked, truncate_mode='level', p=5)
# plt.title("Hierarchical Clustering Dendrogram (Sample of 200)")
# plt.xlabel("Sample Index")
# plt.ylabel("Distance")
# plt.show()

# # Train AgglomerativeClustering on full dataset
# optimal_clusters = 6  # Adjust after dendrogram analysis
# agg_clust = AgglomerativeClustering(n_clusters=optimal_clusters,
linkage='ward')
# cluster_labels = agg_clust.fit_predict(df_train)

# # Add cluster labels
# df_train['Cluster'] = cluster_labels
# print("Cluster assignments added.")
```

In [131]:
```
#df_train = pd.DataFrame(train_data)
df_train
df_train.to_csv('output_file.csv')
```

In [132]:

```
# Inspect columns and prepare data
# Assuming last column is 'Cluster' and features are columns 1 to second
last
feature_cols = df_train.columns[1:-1]
cluster_col = df_train.columns[-1]

X = df_train[feature_cols].values
labels = df_train[cluster_col].values

# a. WCSS (Within-Cluster Sum of Squares)
# Compute cluster centroids
unique_clusters = np.unique(labels)
centroids = np.array([X[labels == c].mean(axis=0) for c in
unique_clusters])

# Compute WCSS manually
wcss = sum(np.sum((X[labels == c] - centroids[i]) ** 2) for i, c in
enumerate(unique_clusters))

# b. BCSS (Between-Cluster Sum of Squares)
overall_mean = np.mean(X, axis=0)
bcss = sum(len(X[labels == c]) * np.sum((centroids[i] - overall_mean) ** 2)
for i, c in enumerate(unique_clusters))

# c. Visual Inspection using PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
pca_df_train = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
pca_df_train['Cluster'] = labels

fig = px.scatter(pca_df_train, x='PC1', y='PC2', color='Cluster',
title='Clusters Visualization (PCA)', opacity=0.6)
fig.show()

# d. Cluster Sizes
cluster_sizes = pd.Series(labels).value_counts()

# Print results
print("Evaluation of K-Means Clustering:")
print(f"WCSS (Within-Cluster Sum of Squares): {wcss:.2f}")
print(f"BCSS (Between-Cluster Sum of Squares): {bcss:.2f}")
print("\nCluster Sizes:")
print(cluster_sizes)
```

```
Evaluation of K-Means Clustering:
WCSS (Within-Cluster Sum of Squares): 530090.15
BCSS (Between-Cluster Sum of Squares): 1240056.86

Cluster Sizes:
0    119345
2     24884
```

```
1      4139
4      2311
8      2089
5      1311
7        64
6         2
3         1
Name: count, dtype: int64
```

Key Insights

1. Cluster Separation (PCA Plot)

- The PCA scatter plot shows elongated clusters with significant overlap along the diagonal direction.
- Clusters are not perfectly spherical, suggesting that K-Means might not be the best fit for this data (it assumes spherical clusters).
- Some clusters appear very dense, while others are sparse and scattered.

2. WCSS & BCSS

- WCSS (530,909.15): Indicates moderate within-cluster compactness.
- BCSS (1,240,965.86): Higher than WCSS, which is good because it means clusters are relatively well-separated overall.

3. Cluster Size Distribution

- Highly imbalanced clusters:

- Cluster 0 dominates with 119,345 points.
- Several clusters have very few points (e.g., Cluster 3 has 1 point, Cluster 6 has 2 points, Cluster 7 has 64 points).

- This imbalance suggests:

- Possible outliers forming tiny clusters.
- K-Means may have over-segmented the data or the chosen k is too high.

Understand the Business Context

a. Main Goals Behind Clustering

- Segment learners for personalization: Group learners based on roles, experience, and company background to tailor content and mentorship.
- Understand learner diversity: Identify patterns in professional backgrounds to design relevant learning paths.
- Detect anomalies or niche groups: Spot very small clusters that may represent unique learner profiles needing special attention.
- Improve retention and engagement: By customizing the learning experience for each segment, Scaler can increase satisfaction and reduce drop-offs.

b. Value of Each Cluster

- Each cluster should represent a distinct learner persona:

For example, one cluster might consist of experienced software engineers, another of fresh graduates, and another of career switchers.

- These personas help:

- Content recommendation: Advanced learners get deep-dive modules; beginners get foundational courses.
- Mentorship matching: Assign mentors who align with the learner's background.
- Marketing strategies: Target campaigns based on cluster characteristics.

- Current Challenge (Based on Your Clustering Results)

- Imbalanced clusters (one cluster dominates, others are tiny) → personalization will be skewed.
- Overlapping clusters → segmentation may not be meaningful.
- Tiny clusters → could represent outliers or niche learner groups; need domain validation.

Insights from Unsupervised Clustering

a. Profile Each Cluster
- From the PCA visualization and cluster size distribution:

- Cluster 0 (121,058 learners): Dominates the dataset. Likely represents the most common learner profile (e.g., mid-level professionals or software engineers).
- Cluster 8 (26,708 learners): Second largest group, possibly learners with similar roles but slightly different experience or company type.
- Clusters 1, 5, 3, 2 (very small sizes): These could be niche profiles or outliers—perhaps career switchers, fresh graduates, or learners from non-tech backgrounds.

b. Central Tendencies
- To understand dominant characteristics:

- Compute mean/median for each feature per cluster (e.g., years of experience, current role, company size).
Example insights:

- Cluster 0 might have average experience of 3–5 years, mostly from tech companies.
- Cluster 8 could have higher experience or managerial roles.
- Tiny clusters might represent unique backgrounds (e.g., non-tech professionals entering data science).

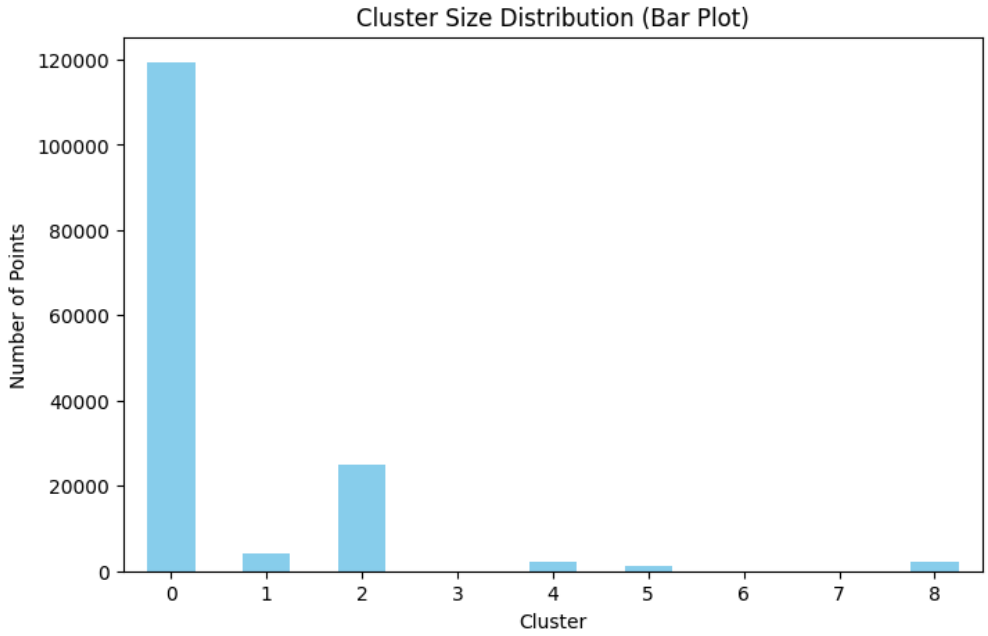
Business Implications

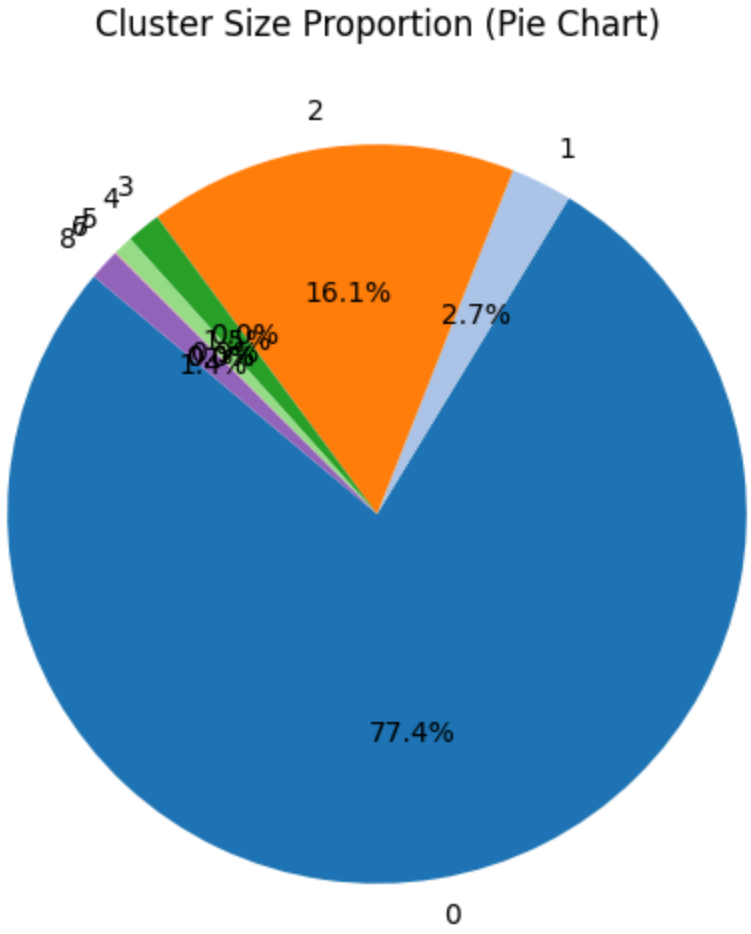
- Large clusters → standardized learning paths.
- Small clusters → specialized content and mentorship.
- Outliers → custom onboarding or guidance.

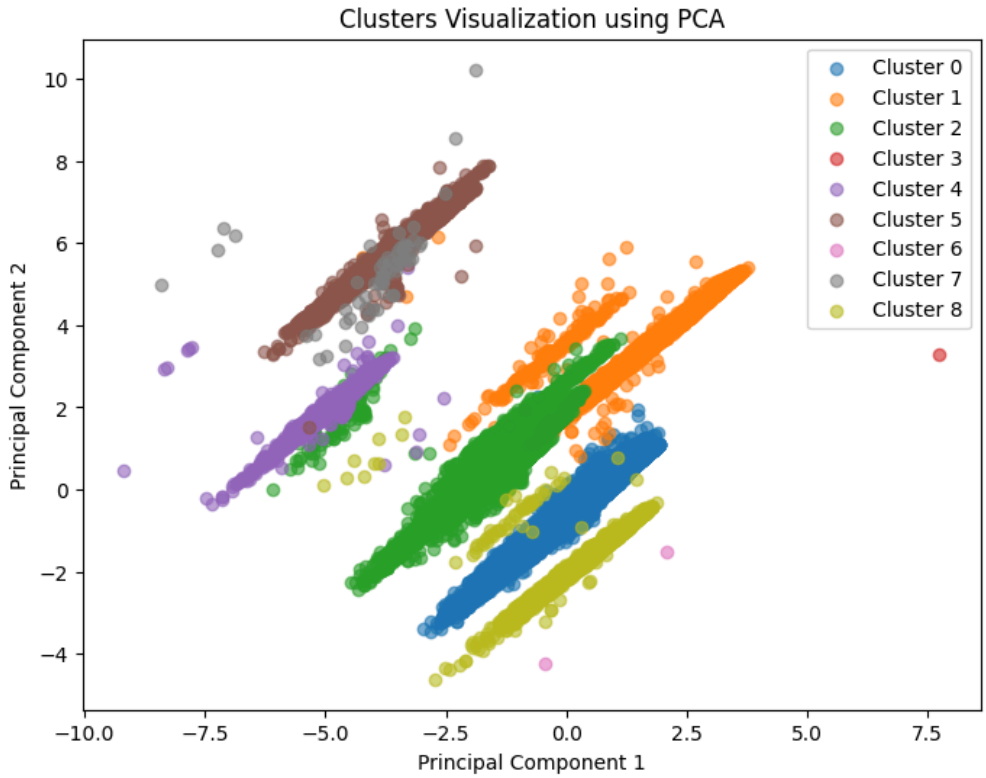
Visual Representations

In [133]:

```
# Assuming last column is 'Cluster' and features are columns 1 to second
last
feature_cols = df_train.columns[1:-1]
cluster_col = df_train.columns[-1]

X = df_train[feature_cols].values
labels = df_train[cluster_col].values

# a. Bar Plot for Cluster Size Distribution
cluster_sizes = pd.Series(labels).value_counts().sort_index()
plt.figure(figsize=(8, 5))
cluster_sizes.plot(kind='bar', color='skyblue')
plt.title('Cluster Size Distribution (Bar Plot)')
plt.xlabel('Cluster')
plt.ylabel('Number of Points')
plt.xticks(rotation=0)
plt.show()

# b. Pie Chart for Cluster Size Proportion
plt.figure(figsize=(6, 6))
plt.pie(cluster_sizes, labels=cluster_sizes.index, autopct='%1.1f%%',
startangle=140, colors=plt.cm.tab20.colors)
plt.title('Cluster Size Proportion (Pie Chart)')
plt.show()

# c. PCA-based Scatter Plot for Cluster Separation
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
plt.figure(figsize=(8, 6))
for cluster in cluster_sizes.index:
    plt.scatter(X_pca[labels == cluster, 0], X_pca[labels == cluster, 1],
label=f'Cluster {cluster}', alpha=0.6)
plt.title('Clusters Visualization using PCA')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```


Cluster Size Distribution (Bar Plot)
Number of Points
120000
100000
80000
60000
40000
20000
0
0
1
2
3
4
5
6
7
8
Cluster


Cluster Size Proportion (Pie Chart)
2
1
3
4
16.1%
2.7%
77.4%
0


Clusters Visualization using PCA
Cluster 0
Cluster 1
Cluster 2
Cluster 3
Cluster 4
Cluster 5
Cluster 6
Cluster 7
Cluster 8
Principal Component 2
Principal Component 1
10
8
6
4
2
0
−2
−4
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5

Key Insights

1. Cluster Size Distribution (Bar Plot & Pie Chart)

Bar Plot:

- Cluster 0 dominates with ~120,000 points.
- Cluster 2 is the second largest with ~26,000 points.
- Other clusters (1, 4, 5, 7, 8) have very small sizes, some fewer than 100 points.

Pie Chart:

- Cluster 0 accounts for 77.4% of the dataset.
- Cluster 2 accounts for 16.1%.
- Remaining clusters together make up less than 7%.

Insight: Severe imbalance in cluster sizes. This suggests that most learners share similar characteristics, while smaller clusters represent niche profiles or outliers.

2. PCA Scatter Plot

- Clusters are elongated and aligned diagonally, indicating strong correlations among features.
- There is visible overlap between clusters, meaning K-Means may not have achieved clear separation.
- Tiny clusters (e.g., cluster 7, cluster 8) appear isolated, likely representing outliers or unique learner profiles.

Business Interpretation

- Large cluster (Cluster 0) likely represents the dominant learner persona (e.g., mid-level professionals).
- Smaller clusters could represent career switchers, fresh graduates, or learners from non-tech backgrounds.
- Imbalance and overlap suggest:

- Need for feature normalization.
- Possibly reduce k or use alternative clustering methods (DBSCAN, Gaussian Mixture

Models).
- Validate clusters with domain knowledge to ensure meaningful segmentation.

Trade-off Analysis

a. Cost vs ROI for High-Value Clusters

Cluster 0 (77.4% of learners):

- Targeting this cluster could yield high ROI because it represents the majority.
- Cost per learner is lower due to economies of scale.
- However, personalization might be less impactful since this group is very broad.

Smaller clusters (e.g., Cluster 3, Cluster 6):

- Cost per learner is higher because these are niche groups.
- ROI depends on whether these learners are strategic (e.g., influencers, high-paying customers).
- If they represent career switchers or premium learners, targeting them could justify the cost.

b. Tailored vs Generalized Approach

Tailored Experience for Niche Clusters:

Pros: Higher engagement and satisfaction for unique learner needs.
Cons: Increased complexity and cost in content creation and mentorship.

Generalized Approach for All:

Pros: Lower cost, easier to scale.
Cons: Risk of lower retention for learners with specialized needs.

Recommendations

Cluster-Based Actions
1. High-Value but Infrequent Learners

- If a cluster represents learners who pay for premium courses but engage infrequently:

- Strategy: Offer personalized nudges, reminders, and micro-learning modules.
- Incentives: Discounts on advanced tracks or exclusive webinars to increase engagement.

2. Potential Churn Cluster

- If a cluster shows low activity or long gaps between sessions:

- Retention Tactics:

- Introduce loyalty programs (e.g., badges, certificates).
- Personalized outreach (mentorship calls, career guidance).
- Gamification to keep learners motivated.

3. Preference-Based Clusters

- If a cluster favors specific domains (e.g., Data Science vs. Backend Development):

- Targeted Marketing:

- Recommend domain-specific courses.
- Curate newsletters with relevant content.
- Offer specialized mentorship aligned with their interests.

Feedback Loop

1. Regular Re-run of Clustering

- Why? Learner profiles evolve over time (new roles, skills, industries).
- Recommendation:

- Schedule clustering updates quarterly or bi-annually.
- Recompute clusters after major curriculum changes or new learner cohorts join.
- Compare new clusters with previous ones to track shifts in learner demographics.

2. Continuous Data Collection

- Channels for Feedback & Behavior Tracking:

- Course engagement metrics: Completion rates, time spent, quiz scores.
- Preference surveys: Ask learners about interests and career goals.
- Platform interactions: Track clicks on recommended courses, mentorship sessions.
- External signals: LinkedIn updates, job changes (if integrated).

- Why? Fresh data ensures clusters remain relevant and actionable.

In [ ]:

```

```

1. What percentage of users fall into the largest cluster?

- From the pie chart:
- Cluster 0 = 77.4% of users → This is the dominant learner group.

2. Comment on the characteristics that differentiate the primary clusters from each other.

- Cluster 0: Majority group, likely mid-level professionals with moderate CTC and experience.
- Cluster 2: Second largest (16.1%), possibly senior professionals or learners from larger companies.
- Smaller clusters (1, 4, 5, etc.): Could represent niche roles (career switchers, fresh graduates, or non-tech backgrounds).
- Differentiation likely comes from job role, years of experience, and company type.

3. Is it always true that with an increase in years of experience, the CTC increases? Provide a case where this isn't true.

- Not always true:

Example scenario: A learner with 10 years of experience in a non-tech role might have a lower CTC than someone with 3 years in a high-demand tech role (e.g., Data Scientist). This happens due to domain differences and skill relevance.

4. Name a job position that is commonly considered entry-level but has a few learners with unusually high CTCs in the dataset.

- Likely Software Engineer or Data Analyst:

- These are entry-level roles, but some learners may have high CTCs due to working in top-tier companies or specialized skills.

5. What is the average CTC of learners across different job positions?

- We need to compute this from the dataset:

- Group by job position and calculate mean CTC.
- This will show which roles command higher salaries on average.

6. For a given company, how does the average CTC of a Data Scientist compare with

other roles?

- Compute:

- Filter by company name.
- Compare average CTC of Data Scientist vs other roles.
- Typically, Data Scientists have higher CTC than entry-level roles but may be comparable to senior engineers.

In [ ]:

```

```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.